

Celfras Standard Protocol

V4.1.0

Celfras Standard Protocol —— 第一部分:标准

本部分规定每一款 Celfras 产品都实现的线路协议(wire protocol):字节在链路上如何成帧、请求如何被应答、核心命令有哪些,以及一个实现要做到什么才算符合 本标准。本部分是自洽的。仅依据本部分即可编写设备端或主机端实现,无需接触任 何产品的源代码。

第二部分是记录,不是要求。它记载各款产品如何填充本标准留给产品自行决定的部 分——标识符映射表、可选命令组、各层自有的错误码。第二部分中没有任何内容放宽 第一部分的任何要求。

1. 范围

1.1 本部分规定的内容

- 符合本标准的链路所运行的传输层(\$2)。
- 帧格式、其 COBS 编码、其完整性校验,以及接收方的校验规则(\$3)。
- 请求/响应模型、状态字节,以及核心错误码空间(\$4)。
- 工作模式,以及每种模式下会被应答的命令(\$5)。
- 核心命令集与各标准化命令组(\$6、\$7、\$8)。
- 操作码空间、其中被保留的部分,以及产品如何扩展它(\$9)。
- 符合性检查表(\$10)。

1.2 本部分刻意不规定的内容

产品之间可以合理存在差异的一切内容:变量命令与参数命令背后的标识符映射表 (\$8)、遥测字段位中哪些承载真实读数(\$7.2)、调校标识符的含义(\$6.4), 以及产品固化的每一个门限值。此外还明确包括传输层的实现——参见 \$2。

1.3 角色

主机(host) —— PC 侧工具。它发出请求,并消费响应与非请求帧。

设备(device) —— 产品固件。它应答请求,并可发出非请求帧。

桥接器(bridge) —— 可选的中继,串接在同一条链路上。它应答保留给自己的 操作码(\$9.2),并把其余一切原封不动地转发给设备。对符合本标准的主机和设备 而言,桥接器是透明的。只有一条规则对它不同,且仅此一条:桥接器不应转发校验 未通过的帧,并可对这样的帧作出应答,而不是静默丢弃(\$3.7)。

一条链路上恰好有一个主机和一个设备参与。桥接器若存在,不改变任一方向上的帧 格式。

1.4 要求用语

应 —— 本标准的绝对要求。违反其中任何一条的实现即不符合本标准。 **不应** —— 本标准的绝对禁止。违反其中任何一条的实现即不符合本标准。 **宜** —— 只有在给出理由时才可以偏离的要求;偏离的产品在第二部分中记录该偏离。 **可** —— 真正可选。

本文中"应""不应""宜""可"四个词只按以上含义使用,承载规范效力;其余动词 均为陈述性表述,不构成要求。

1.5 字节序与记法

除非某个表另有说明,所有多字节字段均为小端(little-endian)且无符号。位 0 为 最低有效位。u8、u16 和 u32 表示相应宽度的无符号整数。字节值以十六进制 书写。

2. 传输层

链路为异步串口:8 位数据位、无校验位、1 位停止位、无流控。默认速率为 1 Mbaud。产品可规定另一速率,但两端应通过带外方式就该速率达成一致——协议本身 不协商它。

链路为全双工,两个方向承载相同的帧格式(§3)。设备可在任意时刻发送非请求帧 (§7);无法容忍这样一帧出现在自己的请求与对应响应之间的主机,不符合本标准。

传输层后端不属于本标准。 设备以 DMA、逐字节中断还是轮询来服务它的 UART, 是实现选择:线路上没有任何东西能区分它们,主机不应依赖于当前使用的是哪一种。 参考实现出厂带 DMA 后端,并带有一个可在编译期选择的中断后端;两者在链路上逐 字节完全相同,且都符合本标准。关于环形缓冲区、发送队列、中断优先级或 DMA 通道 的陈述属于产品自身的文档,绝不属于本标准。

由于 §3.4 的成帧是自同步的,接收方能 在没有带外复位的情况下从被截断或损坏的 帧中恢复,并应接受帧间无空隙、背靠背到达的帧。设备不应要求帧前存在空闲间隔、 线路断开(break)或前导码。

3. 帧格式

3.1 解码后的帧

每一帧,在任一方向上,在去掉 §3.4 的成帧之后具有如下布局:

[LEN u16 LE][CMD u8][SEQ u8][PAYLOAD ...][CRC16 u16 LE]

字段	宽度	含义
LEN	u16 LE	其后跟随的字节数:1 (CMD) + 1 (SEQ) + len(PAYLOAD) + 2 (CRC16)。
CMD	u8	操作码。响应携带与它所应答的请求相同的操作码。
SEQ	u8	关联标记,见 §3.5。
PAYLOAD	LEN - 4	与命令相关,可以为空。对响应而言它总是 [STATUS][DATA...],见 §4。
CRC16	u16 LE	完整性校验,见 §3.3。

解码后的帧总长度为 2 + LEN。因此最短的合法帧是六个字节:LEN(两字节) + CMD + SEQ + CRC16(两字节), 载荷为空。

3.2 LEN 计入什么

LEN 计入 LEN 字段**之后**的一切,其中包括那两个 CRC 字节。它**不计入自 身**。接收方应把 LEN 与自己实际解码出的长度作比较,并应丢弃两者不一致的任何 帧——这项检查是相等判定,不是下限判定。

3.3 完整性校验

CRC16 为 **CRC-16/CCITT-FALSE**:

参数	取值
宽度	16 位
生成多项式	0x1021
初值	0xFFFF
输入反射	否
输出反射	否
最终异或	无

其覆盖范围: 解码后的帧,自 LEN 的第一个字节起至 PAYLOAD 的最后一个 字节止。那两个 CRC 字节本身被排除在外。它在解码后的帧上计算,绝不在 §3.4 的 编码字节上计算。

这是最常见的互操作错误,而且必须同时做对两件互相独立的事:CRC 的变体,以及它 的覆盖范围。先对照下列向量核对变体:

输入	CRC16
31 32 33 34 35 36 37 38 39(ASCII "123456789")	0x29B1
01 00	0x2E3E
(空)	0xFFFF

"123456789" 上得出 0x29B1 是该变体已公布的检验值。得出其他结果的实现用错 了变体——通常的元凶是反射和最终异或——再怎么调整覆盖范围也修不好它。变体确认 无误之后,再用 §3.6 的完整实例核对覆盖范围。

3.4 成帧

解码后的帧经 COBS(Consistent Overhead Byte Stuffing,一致开销字节填充)编码, 其后追加单个 0x00 字节作为帧分隔符:

```
wire bytes = COBS_encode(decoded frame) + 0x00
```

COBS 保证编码后的字节中不含 0x00,因而该分隔符是无歧义的。分隔符**不是编 码数据的一部分,也不在** CRC 的覆盖范围内。

接收方应把 0x00 视为无条件的帧边界:累积字节直到收到一个 0x00,解码已累积 的内容,然后开始新的一帧。正是这一点让链路成为自同步的——发生任何损坏之后, 下一个 0x00 就恢复了帧对齐。接收方应丢

弃长度为零或以其他方式无法解码的累积内容,而不是把它当作一帧。

这里的编码就是 Cheshire 与 Baker 所发表的 COBS,并且其全部内容都陈述于下。符合本标准的实现不需要其他任何资料,尤其**不应**依据"数一数非零字节并发出这个计数"这句概括来编写:那样写出的编码器对短帧正确,对长帧则是错的,而 §3.4.1 的 0xFF 规则正是两者的分野。

3.4.1 编码

编码器遍历解码后的帧,把它切分成若干**分组**。每个分组是以下三者之一:

- n 个非零字节构成的连续段, $0 \leq n \leq 253$, 由解码帧中的一个 0x00 字节 终结;
- 恰好 254 个非零字节构成的连续段,不由任何东西终结;或
- 最后一个由 n 个非零字节构成的连续段, $0 \leq n \leq 253$, 由解码帧的结尾终结。

每个分组发出为一个**码字节**,其后跟随该分组的那些非零字节,顺序不变、内容不改。码字节为 $n + 1$,因此它落在 0x01-0xFF 之内,永远不会是 0x00。解码帧中起终结作用的那个 0x00 **不被**发出:由码字节代表它,零就是这样从线路上被去掉的。

因此码值 0xFF 的含义与其他任何取值都不同:

- 0x01-0xFE —— 其后跟随 $code - 1$ 个非零字节,并且在解码帧中它们之后 **站着一个** 0x00(对最后一个分组而言,则是帧在那里结束)。
- 0xFF —— 其后跟随 254 个非零字节,并且它们之后**没有** 0x00。该分组之所以结束,仅仅因为一个码字节无法计得更多。

编码器应为每一段完整的 254 个非零字节发出一个 0xFF 码,并应随后以一个新的分组继续。切分一段长的无零连续段时,不应引入帧中原本不存在的零。即使最后一个分组不携带任何数据字节,编码器也应为它发出码字节:恰好在 254 字节边界上结束的帧,编码为那个 0xFF 分组后面跟一个孤立的 0x01。

编码后的帧比解码后的帧长一个字节,每出现一个 0xFF 码再多一个字节——对 n 字节的帧最多有 $\text{floor}(n / 254)$ 个。§3.4 的分隔符再加一个字节。

3.4.2 解码

解码器应按如下步骤处理已累积的字节——即自上一个分隔符以来收到的全部内容,不含分隔符本身:

1. 读取一个码字节。码字节为 0x00 时应予拒绝:编码器绝不产生这样的码字节,因此该累积内容不是一帧。
2. 把接下来的 $code - 1$ 个字节原样复制到输出。若剩余字节不足 $code - 1$ 个,应拒绝该帧。
3. 若该码**不是** 0xFF **且**仍有字节剩余,则向输出追加一个 0x00 字节。若该码是 0xFF,或已无剩余,则什么也不追加。
4. 若仍有字节剩余,返回步骤 1。

解码器应拒绝整帧,而不是把一个残缺的帧向上层传递。

步骤 3 恰是那两种码含义的逆运算,而它的两个条件都有分量。在一个 0xFF 分组之后追加零,插入了发送方从未发送过的一个字节。在最后一个分组之后追加零,追加的则是一个原本只是帧结尾的字节。无论哪一种,解码后的长度都会差一个或多个字节,于是该帧通不过 §3.7 的 LEN 检查与 CRC 检查而被丢弃——

这意味着一个写错的解码器表现出来是长帧上的 CRC 失败,而不是解码失败。追查这一症状的实现者宜先检查步骤 3,再去重新核对 CRC。

3.5 SEQ

SEQ 把响应与其请求关联起来。

- 对请求的响应应原样回送该请求的 SEQ。
- 由设备发往主机的非请求帧——不应答任何请求的帧——应使用 $SEQ = 0$ 。
- SEQ 对设备是不透明的。设备不应解释它、校验它,也不应要求它在两次请求之间发生变化。分配 SEQ 取值,以及检出重复或缺口,完全是主机的事。

流水线发出请求的主机应使用 SEQ 匹配响应,并且不应假定响应按请求发出的顺序到达。

由于非请求帧正是以 $SEQ = 0$ 标明自身,在使用流式传输(§7)的链路上主机不宜发出 $SEQ = 0$ 的请求:否则对同一命令而言,由主机发起的停止与由设备发起的停止将无法区分(§7.2)。

3.6 完整实例

本节中的每一个字节都是对照参考实现算出来的,不是示意性的。

3.6.1 一个六字节的请求

一个完整的 CMD_PING 请求, $SEQ = 0x07$, 无载荷。

解码后的帧——六个字节:

04 00 01 07 E7 0D

字节	取值	字段
0-1	04 00	LEN = 4 —— CMD + SEQ + 零个载荷字节 + CRC16
2	01	CMD —— 取自 §6.1 的操作码
3	07	SEQ —— 主机的关联标记;响应回送它
—	—	PAYLOAD —— 本命令为空
4-5	E7 0D	CRC16 = 0x0DE7, 小端

CRC 在字节 0-3(即 04 00 01 07)上计算——包括那两个 LEN 字节,排除那两个 CRC 字节。从 CMD 开始计算的实现会得到另一个值,而它发出的帧会被丢弃且不会收到任何错误应答(§3.7)。

在线路上,经 COBS 编码并加上分隔符之后——八个字节:

02 04 05 01 07 E7 0D 00

字节	取值	含义
0	02	COBS 分组码;其后跟随一个非零字节,并且这里原本站着一个零

字节	取值	含义
1	04	LEN 低字节
2	05	COBS 分组码;其后跟随四个非零字节
3-6	01 07 E7 0D	CMD、SEQ、CRC16 —— 原样承载
7	00	帧分隔符 —— 不是 COBS 数据,不在 CRC 覆盖范围内

注意 COBS 做了什么:解码帧中那唯一的 0x00(LEN 的高字节)被去掉了,取代它 的码字节是 0x02 —— 一个数据字节,然后一个零(\$3.4.1)。最后一个码是 0x05: 四个数据字节,然后是帧的结尾。

3.6.2 一个含 254 字节连续段的帧

上面那一帧无法展示 0xFF 码,因为六个字节里不含长连续段。这一帧可以。它是一个 固件映像数据请求,而这正是实践中出现长无零连续段的地方:映像字节是任意的, 动辄数百个字节都不出现一个零。它的操作码属于保留给桥接层的区段(\$9.2),定义 在第二部分中——本节的成帧在两个区段中完全一致,而这个用例正好把它检验到位。

解码后的帧为 266 个字节:

字节	取值	字段
0-1	08 01	LEN = 264
2	E9	CMD
3	09	SEQ
4-7	00 10 00 00	映像偏移,u32 LE
8-263	01 02 03 ... FD FE 01 02	256 个映像字节;字节 8 + i 存放 $(i \bmod 254) + 1$,故该连续段中没有一个字节的零
264-265	1A 86	CRC16 = 0x861A,小端

编码后为 268 个字节,加分隔符为 269 个,分为五个 COBS 分组:

编码后字节	码	数据字节	该码所表示的含义
0-4	05	08 01 E9 09	四个非零字节,然后一个 0x00 —— 解码字节 4
5-6	02	10	一个非零字节,然后一个 0x00 —— 解码字节 6
7	01	—	没有数据字节,然后一个 0x00 —— 解码字节 7
8-262	FF	01 02 ... FD FE(254 个字节)	254 个非零字节,其后没有零
263-267	05	01 02 1A 86	四个非零字节,然后是帧的结尾
268	—	00	帧分隔符 —— 不是 COBS 数据,不在 CRC 覆盖范围内

因此编码后开头的若干字节是 05 08 01 E9 09 02 10 01 FF 01 02 03 04 ...,而结尾 的若干字节是 ... FB FC FD FE 05 01 02 1A 86 00。

编码字节 8 处的那个 0xFF 才是本例的要点。它的 254 个数据字节是解码字节 8-261;紧随其后的分组从解码字节 262 接续,两者之间什么也没有。在那里插入一个 0x00 的解码器会得到 267 个解码字节而不是

266 个,于是这一帧——它明明是完好 到达的——同时通不过 LEN 检查和 CRC 检查,被丢弃且无任何应答。

也请注意最后一个分组。它的码是 0x05 而不是 0xFF,而且它所携带的四个字节之 后跟的是帧的结尾而不是一个零:解码字节 265 是 CRC 的高字节,其后再无内容。在 最后一个分组之后追加一个隐含零的解码器,错在同一处,也同样差一个字节。

3.7 接收方的校验,按顺序

接收方应按下列顺序施加这些检查,并应在第一项失败时就丢弃该帧:

1. COBS 解码(§3.4.2)成功,且产出至少六个字节。
2. 在解码后的帧上(排除其最后两个字节)计算出的 CRC,等于那两个字节所携带的 CRC16。
3. LEN 等于解码后的长度减二。

任何一项未通过的帧应被丢弃,并且不应被转发。至于可以就它说些什么,则取决于 角色(§1.3)。

设备应静默丢弃它,并且不应应答它。完整性校验未通过的帧,其 CMD 与 SEQ 都不可信,因此既没有可据以应答的操作码,也没有可用来应答的标记;回复只能是猜 测,而且是发给一个也许根本不曾发出过的请求。

桥接器可以用它读到的 CMD 和 SEQ 应答 [ERR][ERR_BAD_CRC],参考桥接器 即如此实现。它的处境不同于设备:一个对自己拒绝转发的帧只字不提的桥接器,与一个收到该帧却将其忽略的设备,在外部看来无法区分,因此作出应答才能把损坏归因于 桥接器所能看见的那一段链路。同样的道理也延伸到唯一职责就是恢复的实现——引导加 载程序(bootloader)——在那里,沉默与一块根本没有启动起来的板子无法区分。

主机应容忍这两种结果:没有应答,以及一个 SEQ 它可能并不认识的 [ERR][ERR_BAD_CRC]。

只有三项检查全部通过之后,该帧才算被接受。无论其操作码最终是什么,正是"接 受"这一事件驱动 §5.3 的链路活跃性计时。

3.8 最大帧长

设备应公布它所接受的最大编码帧长度(不含分隔符);该值是产品常量,记录在第二 部分中。

这个上限是一条真实的边界,不是一条建议。接收方把编码后的字节累积进一个固定大 小的缓冲区,越过其末端的字节被丢弃,于是超长的帧以被截断的形态到达,并按 §3.7 被丢弃——静默地,没有任何错误应答。发出这样一帧的主机什么也观察不到,而这与它 从一条已死的链路上观察到的完全相同。

- 主机不应发送长于设备所公布上限的帧。
- 主机不应以试验方式推断该上限:某一帧得到了应答,只能证明那一帧放得下。
- 设备应接受它所实现的各命令组可能要求它接受的每一帧。设备不应公布一个把它自 己会应答的命令排除在外的上限。

4. 请求、响应与错误

4.1 模型

本协议是请求/响应式的,另带一条非请求的旁路通道。

- 设备处理的每一个请求产生**恰好一个**响应帧,携带相同的 CMD 与相同的 SEQ。
- 设备不对一个请求应答两次,也不应让一个已处理的请求得不到应答——只有一个例外,即 §5.2 的静默丢弃。
- 由设备发往主机的非请求帧(\$7)携带 SEQ = 0,不应答任何东西。
- 设备不要求并发处理请求,主机也不应假定它会。

4.2 状态字节

响应载荷总是 [STATUS u8][DATA ...]。

STATUS	含义
0x00	OK。DATA 是该命令的结果,可以为空。
0x01	ERR。DATA[0] 是一个错误码(\$4.3)。允许有更多的 DATA 字节,其含义随错误码而定。
0x02	保留。符合本标准的设备都不会发出它。

主机应把 0x00 以外的任何状态视为失败,并且不应因一个无法识别的状态值而失去 同步:把该帧读完,报告失败,继续运行。尤其是状态 0x02,它是保留的而不是空闲 的——本协议的一个更早的修订版曾把它规定为第二个错误状态——因此即使现在没有任何 东西发出它,接收方仍须容忍它到达。

设备应发送 [0x01][错误码] 而不是光秃秃的 [0x01]:主机无法把一个光秃秃的错 误与一个被截断的错误区分开。

4.3 核心错误码空间

错误码 0x00 到 0x0F 是**核心空间**。它只在此处定义,别处不得定义,并且它恰 好只包含以下这些码:

Code	Name
0x00	ERR_OK
0x01	ERR_BAD_LEN
0x02	ERR_BAD_CRC
0x03	ERR_BAD_ARGS
0x04	ERR_NOT_READY
0x05	ERR_UNKNOWN

每一个都只有一个含义,而本节文字就是那个含义。

- **ERR_OK(0x00)** —— 无错误。它存在,是为了让零在错误码空间中的含义与 它在状态字节中的含义一致,而且它**从不被发送**:设备报告成功的方式是状态 0x00 且完全不带错误码。[0x01][0x00] —— 一个携带 ERR_OK 的错误状态 —— 是格式非法的响应,主机应把它当作失败而不是成功。
- **ERR_BAD_LEN(0x01)** —— 载荷短于该命令所要求的长度。在检视其中任何取 值之前即已判定。
- **ERR_BAD_CRC(0x02)** —— 某一帧的完整性校验未通过,而收到它的那一层正 在报告这一事实,而不是静默丢弃。设备不应发送此码;桥接器或引导加载程序可以 (§3.7)。
- **ERR_BAD_ARGS(0x03)** —— 载荷长度足够,但其中有内容超出范围、无法识 别,或不可写。它同时也是对设备不予处理的操作码的应答(\$9.1)。

- **ERR_NOT_READY(0x04)** —— 命令被识别,参数也有效,但设备当前的状态不 允许执行它。同样的请求原封不动地稍后再发,可能会成功。它适用于已经打开的会 话(\$6.4)、尚未进入的模式,或主机尚未满足的任何其他前置条件。
- **ERR_UNKNOWN(0x05)** —— 操作码无法识别。可选,对这一情形而言与 ERR_BAD_ARGS 可以互换;参见 \$9.1。

产品不应在 0x00-0x0F 中定义新的码,不应赋予已有的码第二个含义,也不应把某 一个码用于上面所定义之外的情形。这些是主机不必知道对端是什么就能解释的码。

请注意其中最常见的两个码各自的形态。ERR_BAD_LEN 意为载荷短到该命令根本无法 执行,在检视其中任何取值之前即已判定。ERR_BAD_ARGS 意为载荷长度足够,但其中 有内容超出范围、无法识别,或不可写。设备不应对短载荷应答 ERR_BAD_ARGS,也不 应对错误取值应答 ERR_BAD_LEN —— 主机正是靠这一区别把"帧构造错了"和"值选错 了"分开。

命令的载荷长度是一个**下限**,只有 LEN 是按相等判定检查的(\$3.2)。设备不应 因为载荷长于该命令所要求的长度而应答 ERR_BAD_LEN;它应读取该命令所定义的 那些字节,并忽略其余部分。主机不应指望多发的尾随字节会被读取,也不应把一个 OK 响应当作它们被读取的证据:两种情况下回来的是同一个响应,而这正是那些多余字节 无法承载含义的原因。

4.4 扩展空间

0x10 及以上的码构成**扩展空间**,本标准在其中不作任何分配。那里的一个码,含 义由发出它的那一层规定,而且只有在主机知道是哪一层作出的应答之后才可解释。

这并非假想。桥接层(\$9.2)把 0x11-0x15 用于其 SWD 与内核控制操作,把 0x20、0x21 和 0xE1 用于闪存操作,把 0x30-0x35 用于它自身的固件更新。 错误码与操作码是两个不同的空间:这里点名的 0xE1 是一个错误码,而同一个 0xE1 也落在 \$9.2 桥接层保留的**操作码**区段之内这件事,并不说明关于它的任何事情 —— 该 字节在帧中所处的位置决定了它属于哪一个空间。 第一个区块值得精确点名:0x14 和 0x15 是内核暂停与内核恢复,它们紧挨着那些 SWD 错误却并不是 SWD 错误,把它们读作传输故障已经耗掉过实机调试的时间。那些含 义属于桥接器——这里点名的这些区块就是该层分配的全部,而第二部分的桥接器附录载 有会引发它们的那些命令——并且它们对另一层可能赋予同样这些数字的含义什么也没有 说。同一条链路上的一个设备和一个桥接器可以各自把 0x30 用于毫不相干的事情而 不冲突,因为帧本身说明了是它们中的哪一个作出的应答。

产品需要自有错误码时应遵守的规则:

1. 从 0x10 或更高处分配。绝不在核心空间中分配。
2. 在第二部分中把该分配记录在拥有它的那一层名下。
3. 对自己这一层之外的数字不作任何假定 —— 一个码并不因为别的某一层没用它就是 空闲的。

5. 工作模式

5.1 模式

设备任何时候都恰好处于一种工作模式。改变它的是 CMD_SET_MODE(\$6.1)。

Value	Name
0x00	OPMODE_ISP
0x01	OPMODE_NORMAL
0x02	OPMODE_DEBUG
0x03	OPMODE_TUNING
0x04	OPMODE_TEST

本标准规定各模式的取值,并确定每种模式是为什么而设。一种模式在产品内部做什么——启动、挂起还是不理睬哪些子系统——由产品自行规定,并记录在第二部分中。

- **OPMODE_ISP** —— 选择产品的在系统编程通路。实现它的设备在该模式下可以停止响应它平常的输入,并且在没有后续内容到来时可以自行回到 OPMODE_NORMAL。
- **OPMODE_NORMAL** —— 上电默认模式,也是安静的那一个。参见 §5.2。
- **OPMODE_DEBUG** —— 设备所实现的每一个命令都会被应答。
- **OPMODE_TUNING** —— 保留给由主机驱动的标定(§6.4)。
- **OPMODE_TEST** —— 保留给产品自己的产线测试。

设备应以 OPMODE_NORMAL 启动。对于自己没有实现的模式取值,设备应以 ERR_BAD_ARGS 应答 CMD_SET_MODE,而不是忽略它或选中相邻的模式。

设备接受了一种保留模式却并不为之做任何事,这仍然以 OK 确认 —— 该确认的含义是 "取值被识别",不是"这个模式做了什么"。

5.2 NORMAL 应答什么

OPMODE_NORMAL 是上电默认模式,而且它刻意保持安静:在主机主动要求之前,它把 设备自身的时序与主机流量隔离开。

在 OPMODE_NORMAL 下,设备应处理并应答恰好这些命令:CMD_PING、CMD_INFO、CMD_GET_VERSION、CMD_SET_MODE、CMD_DBG_ONLINE 和 CMD_RESET(§6.1、 §6.2)。其他每一个操作码——无论已分配、未分配还是未实现——都应被**静默丢弃**: 完全没有响应,连错误也没有,也不采取任何动作。

这六个就是规范性的集合;别处任何其他的列举都不具权威性。

这是本标准中唯一一处格式正确的请求得不到应答的地方,而且两端必须以同样的方式 理解它。在 OPMODE_NORMAL 下得不到应答的主机,并没有发现一台坏掉的设备;它发 现的是一台还没有人请它离开上电模式的设备。CMD_SET_MODE 在每一种模式下都会被 处理,而且它是唯一的出路。

CMD_PING、CMD_INFO 和 CMD_GET_VERSION 在每一种模式下都会被应答,这是设计 使然:识别一台设备并确认它还活着,不应要求改变它的状态。

5.3 链路活跃性

处于 OPMODE_NORMAL 以外的任何模式时,若在超过链路活跃性间隔 ONLINE_TIMEOUT_MS 的时间内没有任何帧被接受(§3.7),设备应把自己退回 OPMODE_NORMAL。该计时器由**任何**被接受的帧复位,而不是由某一个特定的操作码 复位。

因此,打算停留在 OPMODE_NORMAL 之外的主机应在每一个这样的间隔之内发出一点 什么。
CMD_DBG_ONLINE 作为最廉价的选择而存在——它不带载荷,也不做别的事——但 任何被接受的帧都可以。

其目的不是活跃性上报。而是:非上电默认的模式可能已经挂起了设备自身的某些保护 行为,而一根被拔掉的线缆不可以让这些保护一直挂起。符合本标准的设备应让这一退 回是无条件的:它不能依赖主机确认任何东西,因为所要防范的失效恰恰是主机已经不 在了。

该间隔的取值是产品常量,在第二部分中给出。

6. 核心命令集

本节中的各表由权威的操作码定义生成。请求与响应的形态在表中给出;正文给出围绕 它们的规则。

6.1 身份、模式与活跃性

ID	Name	Request	Response	Notes
0x01	CMD_PING	[]	[OK] ["PONG"]	
0x02	CMD_INFO	[]	[OK] [id string, contains "CFS"]	
0x03	CMD_SET_MODE	[op_mode u8]	[OK] or [ERR] [ERR_BAD_ARGS]	
0x04	CMD_DBG_ONLINE	[]	[OK]	liveness ping, processed in any op_mode
0x05	CMD_GET_VERSION	[sel u8]	[OK] [sel] [major] [minor] [patch]	

本组中的每一个命令都应由每一台符合本标准的设备实现,而且其中每一个在每一种工 作模式下都会被应答。

- **CMD_PING** —— 活跃性探测。它的响应载荷是固定文本;主机宜检查状态字节, 而不是解析那段文本。
- **CMD_INFO** —— 一个供人阅读的身份字符串。它是自由文本,内容由产品自行规 定。主机不应解析它来作决定;CMD_GET_VERSION 就是为此而存在的。
- **CMD_SET_MODE** —— 参见 §5。
- **CMD_DBG_ONLINE** —— 一个空操作,其唯一效果是复位 §5.3 的活跃性计时器。
- **CMD_GET_VERSION** —— 供机器读取的身份。一个操作码通过一个前导选择子字节 回答不止一个问题,而响应回送该选择子,使得一条回复是自描述的:既能从日志里 读懂,也能被流水线发出这两个请求的主机匹配上。

请求为 [sel u8],响应为 [OK] [sel u8] [major u8] [minor u8] [patch u8],选择子 在 DATA[0] 处重复出现。本标准定义两个选择子:

Value	Name	Meaning
0x00	VER_SEL_FW	the firmware build on this board
0x01	VER_SEL_CMD_SET	the opcode table it implements

VER_SEL_FW 报告作出应答的那一份固件映像的版本 —— 这是一个产品身份,在不同产 品之间不可比较。VER_SEL_CMD_SET 报告 §9.3 的命令集版本,本标准的每一个实现都 携带它,而且它在所有实现之间可比 较。两者都是同样的 MAJOR.MINOR.PATCH 形态, 而且谁也无法由谁推导出来。

0x02 及以上的选择子取值本标准未作分配。对于自己没有实现的选择子,设备应应答 ERR_BAD_ARGS,并且 不应用另一个选择子的值来应答 —— 流水线发出了两个请求的主 机,只有被回送的选择子可以把两条回 复分辨开。每一台符合本标准的设备都应实现上 述两个选择子。选择子编号是按操作码各自独立的,并不 共用;桥接层有它自己的版本 命令(§9.2、第二部分),带有它自己的选择子空间,同样的选择子取值在那里表 示别 的东西。

在依赖本组之外的任何操作码之前,主机宜先读取命令集版本。

6.2 设备控制

ID	Name	Request	Response	Notes
0x10	CMD_RESET	[]	[OK] ["SYSTEM RESET"], then NVIC_SystemReset()	
0x11	CMD_ACTIVATE	[]	[OK] (stub)	
0x12	CMD_DEACTIVATE	[]	[OK] (stub)	
0x13	CMD_START_HEATING	[]	[OK][3 bytes] (stub)	
0x14	CMD_STOP_HEATING	[]	[OK][3 bytes] (stub)	

CMD_RESET 是本组中要求每一台设备都实现的命令,而且它在每一种模式下都会被应 答。设备应在复位之 前发出它的响应,并应先让该响应离开发送通路;复位之后没 有任何帧跟随。此后主机应把链路视为已重新 初始化 —— 设备回来时处于 OPMODE_NORMAL (§5.1)。

本组中其余的操作码,是给产品自己的激活控制与加热控制预留的标准化位置。没有实 现其中某一个的产 品应按 §9.1 应答它,而不是予以确认,并且不应把这些操作码中的 任何一个挪作不相干的功能。

一处已知的偏离,在此陈述而不加掩饰。 参考固件今天并不满足那条规则: CMD_ACTIVATE、 CMD_DEACTIVATE、CMD_START_HEATING 和 CMD_STOP_HEATING 无 论产品是否真的为之做事,都应答 [OK],这 就是上表把它们各自标为桩(stub)的原 因。该规则依然成立,新的实现不应照抄这一行为;在它存续期间的 后果是:来自这四 个操作码的 [OK] 不构成任何事情发生过的证据,主机不应把它读作证据。第二部分记 录了哪些产品受此影响。

6.3 寄存器访问

ID	Name	Request	Response	Notes
0x20	CMD_REG_WRITE8	[reg_addr u8][reg_data u8]	[OK][resp_data u8]	
0x21	CMD_REG_READ8	[reg_addr u8]	[OK][resp_data u8]	
0x22	CMD_REG_WRITE16	[reg_addr u8][reg_data u16]	[OK][resp_data u16]	
0x23	CMD_REG_READ16	[reg_addr u8]	[OK][resp_data u16]	
0x24	CMD_REG_BURST_READ16	[reg_addr u8]	[OK][resp_data u16]	

一个可选组,面向对外暴露某个外设或伴随芯片寄存器文件的产品。寄存器地址及其内 容完全由产品自行规定。没有寄存器文件的产品应按 §9.1 应答本组。

6.4 调校

ID	Name	Request	Response	Notes
0x70	CMD_TUNING_START	[tuning_id u8][duration_ms u32][log_fields_mask u16][report_period_ms u16]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	
0x71	CMD_TUNING_REPORT_FRAME		[OK][tick_ms u32][field u16]... (one u16 per set bit in log_fields_mask, LSB first)	(fw->host only, seq=0) same shape family as CMD_LOG_BURST_FRAME
0x72	CMD_TUNING_END	[]	[OK]	host-initiated (echoes req seq) or fw-initiated on duration timeout (seq=0), same idempotent shape as CMD_LOG_BURST_STOP
0x73	CMD_SET_TUNING_PARAMETER	[tuning_id u8][coil_idx u8][heating_mode u8][heater_type u8][step u8][value u16 LE]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	no session required
0x74	CMD_GET_TUNING_PARAMETER	[tuning_id u8][coil_idx u8][heating_mode u8][heater_type u8][step u8]	[OK][value u16 LE] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	no session required

一个可选组,用于由主机驱动的标定:设备在主机控制的时序下运行它真实的量产控制 算法,以便实机测试台可以对其进行测量。本标准确定的是**形态**,不是内容:

- CMD_TUNING_START 打开一个会话;在一个会话已激活时再来一个,应以 ERR_NOT_READY 拒绝。
- 会话激活期间,设备按所请求的节奏以非请求方式发出报告帧,SEQ = 0,采用 §7.2 的字段布局。
- CMD_TUNING_END 关闭一个会话,并且是**双向两用的**,与 CMD_LOG_BURST_STOP 完全一样(§7.2):由主机发起时,它回送请求的 SEQ,并且是幂等的;由设备发起 时——会话时长走完,或者设备自行结束了这次运行——它带着 SEQ = 0 到达。主机应 把两条到达路径当作同一个"会话已结束"事件来处理。
- 设备可以出于自身原因提前结束一个会话。主机不应假定一个会话会跑满它所请求的 时长,也不应假定每个会话至少产出一个报告帧。
- 本组中的参数命令以一个索引元组来寻址一张标定表。每一个索引都应针对它所寻址 的那张表的真实维度作边界检查,越界应为 ERR_BAD_ARGS —— 绝不钳位,也绝不静 默地写入相邻单元。

有哪些调校标识符、每一个扫描什么、索引元组对每一个而言意味着什么,以及一个会话 意味着挂起产品自身的哪些保护行为,全都由产品自行规定,并记录在第二部分中。

6.5 显示

ID	Name	Request	Response	Notes
0x60	CMD_DISP_ON	[]	[OK]	
0x61	CMD_DISP_OFF	[]	[OK]	

一个可选组。没有可由主机控制的显示的产品按 §9.1 应答它。

7. 日志与遥测流式传输

本节中的两种机制都发送由设备发往主机、不应答任何请求的帧。它们全都携带 $SEQ = 0$ (§3.5)。

两个组都是**可选的**。两个都不实现的产品应按 §9.1 应答它们的操作码;本标准中没有别的东西依赖它们。

非请求帧的形态与响应完全相同。 不应答任何请求并不改变它的载荷;按 §4.2 的要求,它是 $[STATUS\ u8]$ $[DATA\ \dots]$,其中 $STATUS = 0x00$,而帧自身的内容从 $DATA[0]$ 开始 —— 也就是从载荷的第 1 个字节开始,不是第 0 个。这对本节中的每一帧以及 §6.4 的调校报告帧都成立。把第一个载荷字节读作内容的主机,在它今后采集的每一个样本上都会错开一个字节,而且这个错误是静默的,因为 $0x00$ 这个状态字节看上去完全可能就是第一个数据字节。生成的各表把这一形态写明了,[OK] 在最前。

7.1 文本日志

ID	Name	Request	Response	Notes
0x30	CMD_LOG_FRAME		[OK][text bytes]	(fw->host only)
0x31	CMD_LOG_START	[]	[OK]	
0x32	CMD_LOG_STOP	[] (host) or (fw->host, unsolicited)	[OK]	

一次面向自由文本诊断信息的拉取式握手,其形态使得设备的日志输出无法淹没链路:

1. 主机发送 CMD_LOG_START。
2. 设备应在武装日志功能**之前**发出它的确认,并应把该确认排在其后一切内容之前,使主机总是先看到确认、再看到任何日志内容。这一顺序是承重的:先武装的设备可能在主机还在等待自己那条响应的时候发出一行日志,而主机会把那一行解析成它所请求的响应。
3. 设备把下一行日志作为一条携带该文本的 CMD_LOG_FRAME 发出,随后跟一条 CMD_LOG_STOP。
4. 主机为下一行重新发出 CMD_LOG_START。

主机应容忍 CMD_LOG_STOP 到达之前没有 CMD_LOG_FRAME:那意味着设备没有什么可说的。主机不应假定该文本以 NUL 结尾,并应以载荷长度为准。

主机还应容忍这样的设备:它在没有再收到 CMD_LOG_START 的情况下持续发出帧 —— 把流限制为一行的正是那一步解除武装,而产品可以刻意让它保持武装以进行连续遥测。需要让流停下来的主机应有办法解除武装:选择 OPMODE_NORMAL 的 CMD_SET_MODE 总是可以做到,而产品也可以把日志功能另外暴露为一个参数槽位(§8)。

文本日志用于诊断。任何以固定节奏采样的东西都应改用 §7.2 —— 在那样的速率下作文本格式化与解析,恰恰是二进制通路存在的意义所在,即为了避免这一开销。

7.2 二进制遥测突发

ID	Name	Request	Response	Notes
0x40	CMD_LOG_BURST_START	[fields_mask u16] [period_ms u16] [duration_ms u32]	[OK] or [ERR] [ERR_BAD_ARGS/ERR_BAD_LEN]	
0x41	CMD_LOG_BURST_FRAME		[OK][tick_ms u32][field u16]... (one u16 per set bit in fields_mask, LSB first)	(fw->host only, seq=0)
0x42	CMD_LOG_BURST_STOP	[]	[OK][total_frame_cnt u16]	host-initiated (echoes req seq) or fw-initiated on duration timeout (seq=0)

一条用于时间序列测量的定宽二进制流。

启动一个会话。 CMD_LOG_BURST_START 接受一个字段位掩码、一个报告周期和一个时长。周期应落在 [BURST_PERIOD_MS_MIN, BURST_PERIOD_MS_MAX] 之内,时长若非 零则应落在 [BURST_DURATION_MS_MIN, BURST_DURATION_MS_MAX] 之内;超出其中任一 范围的取值应以 ERR_BAD_ARGS 拒绝,而不是钳位。

这些是本标准的协议常量,对每一款产品都相同,而它们的取值就在这里:

Name	Value
BURST_DURATION_MS_INFINITE	0
BURST_DURATION_MS_MAX	1000000
BURST_DURATION_MS_MIN	100
BURST_MAX_FIELDS	16
BURST_PERIOD_MS_MAX	1000
BURST_PERIOD_MS_MIN	10

BURST_DURATION_MS_INFINITE 是一个保留的时长取值,不属于时长范围之内:它表示 没有时长,以它启动的会话一直运行到主机把它停下为止(见下文**停止**)。它是低 于 BURST_DURATION_MS_MIN 而设备仍应接受的那一个时长取值。BURST_MAX_FIELDS 是该掩码所能携带的字段位数,这也是该掩码为 u16 的原因。

周期与时长的单位都是毫秒。

字段掩码从标准的 LOG_FIELD_* 系列中选择信号,每个信号一位。这些位的分配 是本标准的一部分:

Bit	Name
0x01	LOG_FIELD_VDD
0x02	LOG_FIELD_VAT
0x04	LOG_FIELD_IAT
0x08	LOG_FIELD_PWR
0x10	LOG_FIELD_DUTY
0x20	LOG_FIELD_PROT

Bit	Name
0x40	LOG_FIELD_RAT

每一位命名一个物理量,而本标准确定的就是这个物理量:

- LOG_FIELD_VDD —— 设备自身的供电轨。
- LOG_FIELD_VAT —— 被驱动负载两端的电压。
- LOG_FIELD_IAT —— 流过它的电流。
- LOG_FIELD_PWR —— 输送到它的功率。
- LOG_FIELD_DUTY —— 驱动的占空比。
- LOG_FIELD_PROT —— 设备的保护状态,以位域形式表示。
- LOG_FIELD_RAT —— 设备算出的负载电阻。

每个字段都以 u16 承载。有两件事由产品自行规定并记录在第二部分中:每个字段背 后的**标度、单位与推导方式** —— 不知道产品的标度就去读取一个值的主机,得到的是一个数字,不是一次测量 —— 以及产品到底把**哪些位接到了真实读数**上。

设备应接受一个自己没有接线的位而不报错,并应把该字段报告为零。高于已分配的最 高位的那些位,本标准未作分配,而设备应把未分配的位完全按照未接线的位来对待: 接受,并报告为零。这两条规则的存在,都是为了让针对较新产品编写的主机不至于在 较旧的产品上失败。

帧布局。 每一条 CMD_LOG_BURST_FRAME 的载荷是 [STATUS][tick_ms u32][field u16] ...:\$4.2 的状态字节,恒为 0x00;然后是一个以毫秒为单位的 u32 时间戳;然后是**所请求掩码中每一个置位的位**各一个 u16,按位号升序排列。次序由位号决定,不由主机选择;未置位的位在帧中不占任何 u16。该时间戳在一个会话之内不应减小,其零点在一个会话之内也不应改变;零点落 在哪里由产品自行规定。掩码不在帧中重复:主机应用它自己发出的那个掩码来解码。设 备不应重排这些值,也不应遗漏其中任何一个。

停止。 CMD_LOG_BURST_STOP 是双向两用的,而这正是调校组所复用的形态 (§6.4):

- *由主机发起* —— 一个请求。设备停止任何激活中的会话,并以该请求自己的 SEQ 应答。它应是幂等的:停止一个并未运行的会话是 OK,不是错误。这是结束一个无限 时长会话的唯一方式,它同时也能提前取消一个有限时长的会话。
- *由设备发起* —— 非请求,SEQ = 0,在一个非零时长走完时发出。

两者携带相同的载荷。主机应能由其中任一者判定自己的会话状态。

并发。 采样不应阻塞命令派发:会话激活期间,普通流量——包括主机自己的停止请 求——必须能够继续流动。把扫描过程放在命令处理函数内部运行的设备无法被停止,不 符合本标准。

8. 变量与参数

ID	Name	Request	Response	Notes
0x50	CMD_VAR8_SET	[var_id u8][value u8]	[OK]	
0x51	CMD_VAR8_GET	[var_id u8]	[OK][value u8]	
0x52	CMD_VAR16_SET	[var_id u8][value u16 LE]	[OK]	

ID	Name	Request	Response	Notes
0x53	CMD_VAR16_GET	[var_id u8]	[OK][value u16 LE]	
0x54	CMD_VAR32_SET	[var_id u8][value u32 LE]	[OK]	
0x55	CMD_VAR32_GET	[var_id u8]	[OK][value u32 LE]	
0x56	CMD_PAR8_SET	[par_id u8][value u8]	[OK]	
0x57	CMD_PAR8_GET	[par_id u8]	[OK][value u8]	
0x58	CMD_PAR16_SET	[par_id u8][value u16 LE]	[OK]	
0x59	CMD_PAR16_GET	[par_id u8]	[OK][value u16 LE]	
0x5A	CMD_PAR32_SET	[par_id u8][value u32 LE]	[OK]	
0x5B	CMD_PAR32_GET	[par_id u8]	[OK][value u32 LE]	

两个命令族提供对设备状态的按索引访问,各有三种宽度。二者的区别在于它们所触及的东西:

- **CMD_VAR*** —— **变量映射表**:实时的运行状态。它主要是对固件实际正在做什么的一个只读视图(状态、测量值、计时器、电量计),而不是它的第二份副本;另有一小组可写的、由主机提供的输入,用来顶替设备本来会从自己硬件上读取的东西。
- **CMD_PAR*** —— **参数映射表**:设备的配置与门限存储。

本组是**可选的**。两个族都不实现的产品应按 §9.1 应答全部十二个操作码。产品可以只实现其中一族而不实现另一族,也可以只实现其中一种宽度而不实现其他宽度——但已实现的某一种宽度应同时实现它的 SET 与它的 GET,因为一个能写却读不回来的值无法被核对。

两个族共用同一种形态。SET 接受 [id u8][value];GET 接受 [id u8] 并应答 [OK][value];该值按命令的宽度为 u8、u16 或 u32,小端。这三种宽度是三个互相独立的标识符空间——8 位空间的 id 3 与 16 位空间的 id 3 是不相干的两个槽位。

要求的行为:

- 达到或超过该宽度表大小的 id 应被应答 ERR_BAD_ARGS,而且是在尝试任何派发之前。
- 对只读槽位的 SET 应被应答 ERR_BAD_ARGS。它不应存储该值,也不应应答 OK——主机无法把一次被静默忽略的写入与一次成功的写入区分开。
- 符合本标准的**设备**应公布它对每一个可写槽位所接受的取值范围。这个范围是设备对自身的陈述:接受其宽度所能容纳的一切取值的槽位,就公布这一点;接受得更少的槽位,就公布它所接受的那些。本标准不规定该公布放在哪里、采取什么形式——数据手册、产品手册,或者第二部分那种形式的附录都可以——只要求设备对自身取值范围的陈述确实存在,并且是主机所依据的那一份。主机工具在自己那份标识符映射表副本里保留的限制,是那个工具的输入便利设施,运行它的人可以调整,并不是设备的取值范围——主机不应把它当作设备的取值范围。
- 携带一个槽位并不接受的取值的 SET 应被应答 ERR_BAD_ARGS。设备不应对该值作掩码处理或钳位,也不应对一个自己随后就丢弃的取值应答 OK:请求了不可能之事的 主机是有缺陷的,而上述三种结果中的每一种都递给它一批看似合理的读数,取代了本可以让该缺陷浮出水面的那个错误。
- 对同一槽位成功 SET 之后再对它发出的 GET,应返回被设置的那个值,除非设备自己在此期间改变了它。这是该规则在线路上可观察的形式,也是主机唯一能核对的形式:值存放在哪里是设备自己的事,但一次读不回来的写入,与一次被丢弃的写入无法区分。做不到这一点的槽位应以 ERR_BAD_ARGS 拒绝 SET,并应被声明为只读。

最后这两条,从线路上看是同一条规则,而且它就是主机所能观察到的全部:在一次被应答为 OK 的 SET 之后,对该槽位的 GET 返回被设置的那个值;而不肯接受某个取值的设备,把这件事说出来,而不是应答 OK。不存在符合本标准的第三种结果——没有哪个 OK 可以意味着设备留下的是别的东西,也没有哪个 OK 可以意味着它什么也没留下。

这条规则所禁止的那种失效很容易在无意中造出来,而且值得点名,因为它根本不是一个协议层面的错误。当某个槽位的设置函数在设备内部拒绝了一个取值,却没有办法把这次拒绝报告回命令处理函数时——一个返回 void 的设置函数是常见的形态——这次写入就被丢掉了,而处理函数因为无可报告,就应答了 OK。在某次 GET 与最后写入的值不一致之前,线路上没有任何东西与之矛盾,而一台从不回读该槽位的主机则根本不会发现这件事。那次拒绝必须到达组装响应的那一层,而那一层必须发出 ERR_BAD_ARGS。

8.1 标识符映射表由产品自行规定,而这一点是承重的

本标准定义命令、宽度以及出错行为。它不定义哪个槽位号表示哪个物理量。那些映射表按产品分别指派,并记录在第二部分中。

它们在产品之间不可移植,而且协议无法防范用错映射表。在一款产品上有效的槽位号,极有可能在另一款产品上同样有效,却寻址到一个完全不同的字段——于是设备执行了错误的写入,并应答 OK。任何一层都没有可供观察的错误。因此:

- 主机应在发出任何 VAR/PAR 访问之前先确定自己在与哪一款产品、哪一个命令集版本对话 (CMD_INFO、CMD_GET_VERSION),并应使用那款产品自己的映射表。
- 产品应以追加的方式指派新槽位。给已有槽位重新编号会改变一台未作改动的主机所写入的对象,属于命令集的 MAJOR 变更(\$9.3)。

8.2 两个操作码触及同一字段时,它们应给出一致的答案

本标准定义了不止一条通往设备可配置状态的路径:上面的参数族,以及在实现了调校组的产品上,该组自己的参数命令(\$6.4)。产品可以通过其中不止一条路径暴露同一个底层字段。

一旦如此,那些操作码对该字段应给出一致的答案。它们应就该字段是否可写达成一致,并应对它接受相同的取值。

没有这条规则,一款产品就可以在同一个编译产物中对同一个字段给出两种不同的答案:通过一个操作码写入被接受并应答 OK,而完全相同的写入通过另一个操作码却被 ERR_BAD_ARGS 拒绝。主机无从化解这一矛盾。就各自的前提而言,两个答案都不算错——每个处理函数施加的都是另一个并不知道的规则——而帧中没有任何东西说明哪一个描述了这台设备,于是主机只能通过第三条路径回读来判定该字段是否接受了那个取值。

可写性是字段的属性,不是主机恰好经由的那个操作码的属性,而主机有权确定它一次就处处依赖它。想让某个字段只能由一个操作码触及的产品,应以不把它映射进另一个操作码来表达这一点,而不是映射进去再在那里拒绝它。

9. 操作码空间、保留与演进

9.1 未分配与未实现的操作码

在 OPCODE_NORMAL 之外,设备应以 [ERR][ERR_BAD_ARGS] 应答它不予处理的**每一个**操作码。这同等地涵盖两种情形:本标准尚未分配的操作码,以及本标准已分配但 本产品没有实现的操作码。沉默不是符合本标准的答案 —— 主机必须能够把一条不受支持 的命令与一条已死的链路区分开。

唯一的例外是 OPCODE_NORMAL 的静默丢弃行为(\$5.2)。

ERR_UNKNOWN 正是为此而存在于核心错误码空间中,能够把无法识别的操作码与错误参 数区分开的设备可以使用它。主机应把这两个码都接受为"这台设备不会做那件事"。

9.2 区段保留

操作码字节被划分为两个区段,而正是这一划分让一条物理链路能够承载两场对话:

区段	归属
0x00-0xBF	设备(产品固件)。新命令在其中向上生长。
0xC0-0xFF	保留给桥接层。 产品不应在此处定义命令。

桥接器仅依据操作码字节路由:在 0xC0 及以上它自己应答该帧,在 0xC0 以下它把 原始的编码字节原封不动地转发给设备。因此,定义在 0xC0 或以上的产品命令,只要 链路中串接了桥接器就永远无法到达 —— 而且这种失效是静默的。没有可供观察的错误: 桥接器把该帧吃掉并应答了它,设备从未见到它,而主机从错误的设备那里得到一条看似 合理的回复。

该保留是整个层共用的一个区段,不是某一块板子的区段。每一个桥接器实现都声明同一 张表。

在设备自己的区段之内,操作码是分组的,产品宜把新命令放进与其功能相匹配的组里:

基址	组
0x00	身份、模式、活跃性(\$6.1)
0x10	设备控制(\$6.2)
0x20	寄存器访问(\$6.3)
0x30	文本日志(\$7.1)
0x40	二进制遥测(\$7.2)
0x50	变量与参数(\$8)
0x60	显示(\$6.5)
0x70	调校(\$6.4)

9.3 版本管理

命令集版本是一个 MAJOR.MINOR.PATCH 三元组,命名本标准的操作码映射 —— 整 张映射,两个区段都包括在内。它不是任何固件的构建号,而且它与固件构建号各自独立 地推进。

- 当某个已有操作码改变了它的编号或含义时, **MAJOR** 变更。否则, 针对较旧那张表 编写的主机会到达另一个处理函数并得到一个看似合理的答案, 而这恰恰是版本号存在 的意义所在: 让这种失效可被检出。
- 当有操作码被新增, 而所有已有操作码的含义保持不变时, **MINOR** 变更。
- **PATCH** 用于对线路没有影响的变更。

有两条不同的规则使用这个号码, 而把二者混为一谈, 正是一套真实系统因为一个根本不 是故障的差异而被拒之门外的原因。

发布记账 —— 一个构建产物的属性。 在同一个实现的同一次发布之内, 命令集版本 常量的每一份副本都应携带同一个取值, 而且在它变更时每一份副本都应一同变更。这个 号码在不止一个代码仓库中的多份固件与主机工具里重复出现, 因此这些副本必须逐一数 清, 而不能想当然; 第二部分为它所涵盖的产品列出了它们。一次发布若为自己的命令集 版本发出了两个不同的取值, 即为构建有误。这条规则的任何方面在线路上都不可观察。

线路兼容性 —— 一条链路的属性。 同一条链路上的两个实现可以报告不同的命令集 版本, 而这一差异本身并不是故障。报告较旧版本的设备, 是在如实陈述它是针对哪一张 操作码表构建的, 而这正是值得掌握的事实。主机:

- 在依赖 \$6.1 之外的任何操作码之前, 宜读取对端的版本 (CMD_GET_VERSION 配以 VER_SEL_CMD_SET, \$6.1);
- 在出现 MAJOR 差异时宜拒绝继续, 因为某个操作码可能已在它脚下改变了编号或含义, 而得到的答案看上去都很合理;
- 在出现 MINOR 或 PATCH 差异时可以继续, 并且此时宜把自己限制在两个版本中较低的那个所定义的操作码之内。按定义 MINOR 是追加式的, 因此较旧的那一端拥有的命令 更少, 而不是不同。

10. 符合性检查表

一台符合本标准的设备, 对下列每一项适用于它的条目都回答“是”。

某一条目不适用时。 有五个命令组是可选的: 寄存器访问(\$6.3)、调校(\$6.4)、显示(\$6.5)、流式传输(\$7), 以及变量与参数(\$8)。检验某个设备没有实现的组的 条目, 回答为**不适用**, 而带有不适用条目的设备仍然符合本标准 —— 前提是第 25 条 对该组的那些操作码成立, 正是它使得“未实现”成为主机可以自行观察到 的事, 而不是 必须由别人告知的事。传输与成帧、请求与响应、模式这三节下的条目, 以及第 22、23、24、25、26、27 条, 任何时候都不会不适用: 那些是对每一台设备的要求。

传输与成帧

条 目	要求	参见
1	串行链路为 8 位数据位、无校验位、1 位停止位、无流控, 速率为双方约定的速率。	2
2	不要求主机提供帧间空隙、线路断开或前导码, 并且接受背靠背的帧。	2
3	每一帧都经 COBS 编码并带一个尾随的 0x00 分隔符, 接收时把 0x00 视为无条件的帧边界。	3.4
4	编码时, 每一段完整的 254 字节无零连续段都以一个 0xFF 码发出, 不在原本没有零的地方引入 0x00, 而且最后一个分组即使没有数据也带一个码字节。	3.4.1

条目	要求	参见
5	解码时,在每一个码不是 0xFF 的分组之后恢复一个 0x00,而在其他任何分组之后都不恢复 —— 包括帧中的最后一个分组。	3.4.2
6	LEN 计入 CMD + SEQ + 载荷 + CRC,并与解码后的长度按相等判定核对。	3.2
7	CRC 为 CRC-16/CCITT-FALSE,并在 "123456789" 上重现 0x29B1。	3.3
8	CRC 覆盖解码后的帧,自第一个 LEN 字节起至最后一个载荷字节止,并排除 CRC 字节本身。	3.3
9	COBS 解码、CRC 检查或 LEN 检查未通过的帧被静默丢弃,不作任何响应。	3.7
10	所接受的最大编码帧长度已公布,而更长的帧像任何其他无法通过校验的帧一样被静默丢弃。	3.8

请求与响应

条目	要求	参见
11	每一个被处理的请求产生恰好一个响应,回送它的 CMD 与它的 SEQ。	4.1
12	非请求帧携带 SEQ = 0。	3.5
13	设备从不解释或校验 SEQ。	3.5
14	每一个响应载荷都以一个状态字节开头,而错误响应在 DATA[0] 中携带一个错误码。	4.2
15	每一个核心错误码只用于 §4.3 为它定义的那一种情形,而任何产品自有的码都在 0x10 或以上。	4.3、4.4
16	ERR_BAD_LEN 用于短载荷,ERR_BAD_ARGS 用于错误取值,二者绝不互换。	4.3
17	长于命令所要求长度的载荷被接受且其多余部分被忽略,绝不应答 ERR_BAD_LEN。	4.3

模式

条目	要求	参见
18	设备以 OPMODE_NORMAL 启动。	5.1
19	CMD_SET_MODE 在每一种模式下都被处理,而未实现的模式取值以 ERR_BAD_ARGS 拒绝。	5.1
20	在 OPMODE_NORMAL 下,CMD_PING、CMD_INFO、CMD_GET_VERSION、CMD_SET_MODE、CMD_DBG_ONLINE 和 CMD_RESET 被应答,其他一切被静默丢弃。	5.2
21	任何非默认模式在 ONLINE_TIMEOUT_MS 内没有帧被接受之后,无条件退回 OPMODE_NORMAL。	5.3

命令

条目	要求	参见
22	§6.1 中的每一个命令都已实现,并在每一种模式下被应答。	6.1
23	CMD_GET_VERSION 实现了 VER_SEL_FW 与 VER_SEL_CMD_SET 两者,回送收到的选择子,对其他任何选择子应答 ERR_BAD_ARGS,并在 VER_SEL_CMD_SET 下报告 §9.3 的版本。	6.1、9.3
24	CMD_RESET 在复位之前作出响应。	6.2
25	在 OPMODE_NORMAL 之外,设备不予处理的每一个操作码都被应答 [ERR][ERR_BAD_ARGS],或者在设备能够把无法识别的操作码与错误参数区分开时被应答 [ERR][ERR_UNKNOWN]。	9.1

条目	要求	参见
26	没有命令定义在 0xC0 或以上。	9.2
27	本次发布中命令集版本常量的每一份副本都携带同一个取值。	9.3

流式传输 —— 对既不实现 §7.1 也不实现 §7.2 的设备不适用。

条目	要求	参见
28	日志帧与遥测帧以 SEQ = 0 发出。	7
29	非请求帧的载荷以状态字节开头,而帧自身的内容从 DATA[0] 开始。	7、4.2
30	CMD_LOG_START 在日志功能被武装之前得到确认,而该确认排在任何日志内容之前。	7.1
31	超出各自允许范围的突发周期或时长以 ERR_BAD_ARGS 拒绝而不是钳位,并且 BURST_DURATION_MS_INFINITE 被接受。	7.2
32	突发帧为掩码中每一个置位的位携带一个 u16,按位序升序排列,而未接线或未分配的位报告为零而不是报错。	7.2
33	停止或结束命令是幂等的,由主机发起时回送请求的 SEQ,并且在设备自行结束会话时也以 SEQ = 0 非请求地发出。	6.4、7.2
34	流式会话激活期间,命令派发保持工作。	7.2

变量与参数 —— 对两族都不实现的设备不适用。

条目	要求	参见
35	超出范围的标识符以 ERR_BAD_ARGS 拒绝。	8
36	对只读槽位的写入以 ERR_BAD_ARGS 拒绝,且什么也不存储。	8
37	设备公布它对每一个可写槽位所接受的取值范围,而它不接受的取值以 ERR_BAD_ARGS 拒绝 —— 绝不钳位、绝不作掩码处理,也绝不应答 OK 之后丢弃。	8
38	对同一槽位成功 SET 之后的 GET 返回被设置的那个值。	8
39	槽位编号仅追加,任何重新编号都伴随一次命令集 MAJOR 版本变更。	8.1、9.3
40	一个字段可经由不止一个操作码触及时,那些操作码就它是否可写以及它们所接受的取值达成一致。	8.2

一台符合本标准的主机还应对下列各条回答"是"。

条目	要求	参见
41	它按 SEQ 匹配响应,并且不假定请求的顺序。	3.5
42	它容忍非请求帧在任意时刻到达,包括在它自己的请求与对应响应之间。	2、7
43	它容忍一个无法识别的状态字节而不丢失帧同步。	4.2
44	它把非请求帧的第一个载荷字节读作状态字节,并从 DATA[0] 取该帧的内容。	7
45	对于损坏到达的帧,它容忍两种答案:来自设备的沉默,以及来自桥接器或引导加载程序的 [ERR][ERR_BAD_CRC]。	3.7
46	它不发送大于设备所公布最大值的帧,也不以试验方式确定该最大值。	3.8
47	它在使用任何标识符映射表之前先读取设备的身份与命令集版本,并使用那款产品自己的映射表。	8.1

Celfras Standard Protocol —— 第二部分:产品

第一部分是标准。本部分是各款产品如何运用第一部分留给它们的余地的记录:§8 背后 的标识符映射表、§3.8 与 §5.3 声明为由产品自行规定的那些常量的取值、产品实现了 哪些可选命令组,以及产品的行为在哪里偏离了第一部分的要求。

本部分中没有任何内容放宽第一部分的任何要求,也没有任何内容是对新实现的要求。一篇附录陈述的是关于某一款产品在某一时刻的事实。当这样一条事实被证明是每一个实现者都需要的时,它就属于第一部分,并被移入第一部分,而不是从此处引用。

本部分承载什么,以及不承载什么。 它承载结构:标识符、名称、单位、访问权限, 以及每个槽位是干什么用的。它不承载产品写进这些槽位里的数字。标准定义标识符 14 表示什么;某一款产品在那里存放了什么,是那款产品自己的事,它承载在本文档的内部 版本以及该产品自己的文档中。内部版本会为每个槽位增设一列给出其内置取值,本版本 则干脆没有那一列,而且本文中任何地方都不出现门限值。

那条规则针对的是标识符映射表的内容。它不是一条针对一切数字的规则:凡是第一部分 要求设备公布它自身的某个常量,以便主机能够与它通信的地方 —— §3.8 的最大编码帧 长、§5.3 的链路活跃性间隔 —— 其取值都在此处给出,因为主机没有它就用不了这条链 路。本部分中出现的一个光秃秃的数字,应读作这类链路常量之一,绝不可读作某张映射 表里的门限值。

本部分论及某个槽位时,只称呼该槽位的名字,而把编号留给它上方的那张表。那些表是 从产品本身生成的;把某个 id 重新敲进一句话里,就等于把同一个事实存了第二份副本, 而本部分已经经历过一次重新编号了(A.2)。

A. CFS-ECIG-SUITE

第一部分的参考实现,也是本文档中每一张生成的标识符表所提取自的那款产品。

A.1 本产品实现了什么

产品常量。 第一部分把其中两个交给第二部分:

- 所接受的最大编码帧长度,不含分隔符(§3.8),在出货的 CWM2032 构建上为 **32 字节**。累积缓冲区是在同一份固件之内按芯片分别定尺寸的,因此换一颗芯片 构建出来的就是另一个数字;主机面对的是一块板子,不是一棵源码树,而这里给出的 是那块板子的数字。
- 链路活跃性间隔 `ONLINE_TIMEOUT_MS`(§5.3)为 **5000 ms**。

可选组。 它实现了文本日志(§7.1)、二进制遥测(§7.2)、调校(§6.4),以及 变量族与参数族在全部三种宽度上的实现(§8)。它**没有**实现寄存器访问(§6.3)或 显示(§6.5):那些操作码在它的头文件中有声明,但没有任何东西派发它们,因此它们 像设备不予处理的任何其他操作码一样按 §9.1 被应答。

§6.2 的那处偏离就是这款产品的。 `CMD_ACTIVATE`、`CMD_DEACTIVATE`、`CMD_START_HEATING` 和 `CMD_STOP_HEATING` 就是第一部分 §6.2 所点名的那些桩:无论有没有事情发生,它们都应答 [OK]。来自这四个操作码的 [OK] 不构成设备已经动 作过的证据。

在这里 `CMD_SET_MODE` 还会驱动日志参数。 进入 `OPMODE_NORMAL` 或 `OPMODE_DEBUG` 会分别强制把 A.3 的 `LOG_ENABLE` 参数槽位置为关闭和打开,因此进 入 `OPMODE_DEBUG` 之后的第一行日志无需 `CMD_LOG_START` 就

会到达。这是 2026-07-10 作出的一个刻意选择,也是附录 B 拿来作对照的那一处行为。

A.2 变量映射表

8 位空间:

ID	Name	Unit
0	EXT_CTRL_STATUS	EXT_CTRL_* bits
1	EXT_HEATING_PWM_DUTY	%
2	OP_MODE	OPMODE_*
3	STATE	STATE_E
4	OIL_MODE	heating profile idx
5	COIL_SEL	H_COIL*_MASK
6	EVENT_FLAG	EVENT_MASK_*
7	EVENT_TOGGLE_FLAG	EVENT_MASK_*
8	BATTERY_GUAGE	%
9	OIL_GUAGE	%
10	EVENT_EMUL_MASK	EVENT_MASK_*
11	EVENT_EMUL_ENABLE	0/1

16 位空间:

ID	Name	Unit
0	EXT_HEATING_POWER	mW (CP) / mV target_vatrms (CV)
1	PROT_TYPE	PROTECT_MASK_*
2	CHARGING_TIME	min
3	VAT	mV
4	IAT0	mA
5	PAT0	mW
6	RAT0	mohm
7	DUTY0	%

32 位空间:

ID	Name	Unit
0	STATE_TIME	ms
1	DISP_TIME	ms
2	ONLINE_ELAPSED	ms
3	HEATING_TIME	ms

ID	Name	Unit
4	LED_DAISY_STARTED	bursts
5	LED_DAISY_DONE	bursts

这张映射表的大部分,正是 §8 所描述的、对固件实时状态的只读视图:状态机的当前状态及其计时器、设备正在进行的测量、它正在计算的各种电量估计。对其中任何一项的 SET 都应答 ERR_BAD_ARGS。

有五个槽位可写,而其中每一个都是主机提供的一个**输入**,用来顶替固件本来会从自己硬件上读取的东西——绝不是设备控制状态的第二份副本。这样一来,量产控制代码就可以原样运行,而不必长出只为主机而设的分支。

- EXT_CTRL_STATUS、EXT_HEATING_PWM_DUTY 和 EXT_HEATING_POWER 构成一个外部控制接口;它的各位表示什么、每个槽位接受什么、主机在驱动它时必须做什么,都在该产品自己的文档中,不在此处。
- EVENT_EMUL_MASK 和 EVENT_EMUL_ENABLE 让主机得以顶替设备自己的事件输入。两者都接受其槽位的全宽取值。

主机在信任某个读数之前还需要知道另外三件事:

- **16 位映射表中的测量部分假定只有一根加热线圈。** 那些按线圈划分的槽位以线圈 0 命名,因为这块板子只有那一根;双线圈的板子会在它们之后追加自己的线圈 1 槽位,而不是给这些重新编号。
- **某个槽位读到的可能是零而不是一次测量,这取决于构建配置。** 在恒压构建上 RATO 没有电流采样通道可读,而在菊花链显示构建上 DISP_TIME 没有按状态划分的时间线。那里的零表示本构建未接线,与它对一个未接线的遥测位所表示的含义完全一样(§7.2);它不是一次读数为零的测量。
- **变量编号在迄今造出的各块板子上恰好相同,而这是巧合而非保证。** §8.1 适用于这张映射表,正如它适用于参数映射表:这种一致是历史的偶然,它对下一块板子什么也不承诺,也不能使任何事情变得安全。主机在使用其中任何一张映射表之前,仍然要先确定自己在与哪一款产品对话。

这张映射表曾在 2026-07-29 被重新编号过一次,当时外部控制区块是插到 8 位与 16 位空间的最前面,而不是追加到末尾。那件事发生在附录 E 的命令集编号之前,因此没有任何版本差异标记它;§8.1 的仅追加规则自那以后一直成立,而一份较旧的映射表副本会寻址到错误的槽位。

A.3 参数映射表

8 位空间:

ID	Name	Unit	Access
0	LOG_ENABLE	0/1	rw

16 位空间:

ID	Name	Unit	Access
0	VDD	mV	ro
1	VATO	mV	ro
2	IATO	mA	ro

ID	Name	Unit	Access
3	LED_BREATH_PERIOD_MS	ms	rw
4	UVLO_HEAT_INIT_TH	mV	rw
5	UVLO_HEAT_TH	mV	rw
6	LONG_PUFF_TH	ms	rw
7	CHG_OVP_TH	mV	rw
8	CHG_TIMEOUT_TH	min	rw
9	SHORT_COIL1_TH	mV	rw
10	SHORT_COIL2_TH	mV	ro
11	SHORT_COIL_RST_TH	mV	rw
12	OPEN_COIL1_TH	mV	rw
13	OPEN_COIL2_TH	mV	ro
14	DRY_PUFF_ABS_TEMP_TH	degC	rw
15	DRY_PUFF_REL_SLOPE_TH	degC/sec	rw

32 位空间:

ID	Name	Unit	Access
0	HEATING_TIMEOUT_TH	ms	rw

这些编号是这款产品的,而且它们不可移植。 这是第二部分中后果最严重的一个事实,而 §8.1 陈述了它背后的规则。每一块板子都独立地为自己的参数枚举编号,因此同一个 id 在另一款产品上寻址的是另一个字段——而且没有任何东西能检出这件事。指向错误板子的主机读到的是错误的测量值,更糟的是,它写入的是错误的门限:固件会执行那次写入,并应答 **OK**。任何一层都没有错误,没有对不上的校验和,事后日志里也找不到任何线索。唯一的防线就是 §8.1 所要求的那一条——先读取身份与命令集版本,并使用那款产品自己的映射表。

大多数可写槽位只在调校构建中才是开放的。 那些保护门限——16 位与 32 位映射表中除 LED_BREATH_PERIOD_MS 之外的每一个 rw 槽位——写入的是设备的参数块,而在发布构建中那个块是 const 的。在那里,它们中的每一个都应答 ERR_BAD_ARGS 而不是存储。这是出货单元的预期行为,不是故障,也不是回归:发布构建就是门限已经等于产品标定结果的那种构建。上面各表给出的是各槽位在调校构建中的访问权限,而调校构建正是其映射表值得记录的那一个。有两个可写槽位不受影响,因为它们根本不触及参数块:LOG_ENABLE 和 LED_BREATH_PERIOD_MS 经由服务调用生效,在每一种构建中都保持可写。

有两个槽位是为这块板子并不具备的第二根加热线圈保留的。 声明 SHORT_COIL2_TH 和 OPEN_COIL2_TH,是为了让编号已经与双线圈构建对得上。它们在每一种构建中都读作零并拒绝每一次写入——这不是上面那种调校构建效应,选择调校构建也不会打开它们。

关于个别槽位还有四件较小的事实:

- SHORT_COIL1_TH 与 OPEN_COIL1_TH 是一对,固件是把它们互相比,而不是各自与一个固定界限比较:两者针对同一个实测压降作检验,低于其中一个门限为断路故障,高于另一个门限为短路故障。

因此它们的大小次序构成了对主机可写取值的一项约束——短路门限应始终高于断路门限。二者交叉之后,它们之间那条健康带就变成两种故障同时触发的带,于是交叉它们不是收紧了线圈检测,而是把它废掉了。

- CHG_OVP_TH 与 SHORT_COIL_RST_TH 可读可写,但目前没有任何东西拿它们作比较——一个是一路分压读数,其比较已被禁用,另一个所属的保护是靠别的手段检出的。保留它们是为了编号的稳定性,今天写入它们不改变任何东西。
- LED_BREATH_PERIOD_MS **接受一个它并不存储的零**:零会在显示斜坡中造成除零,因此设置函数提前返回,而命令仍然应答 [OK],随后的一次 GET 返回先前的值。这是对 §8 回读规则的一处偏离,在此如实记录而不加掩饰。它接受的取值范围是一及以上,新的实现应对那个零应答 ERR_BAD_ARGS,而不是照抄这一行为。
- LOG_ENABLE **存储的东西与收到的东西不同**:它的设置函数把任何非零取值都压成一,而命令一概应答 [OK]。因此对任何非零取值的 SET 都被应答 OK,而随后的一次 GET 返回一。这正是 §8 所禁止的掩码处理,而且它与上一个槽位是同一个缺陷的另一面——一个无法把拒绝报告给组装响应那一层的设置函数,于是响应说 OK。它接受的取值范围是零和一,新的实现应对其他任何取值应答 ERR_BAD_ARGS,而不是照抄这一行为。这两处是同一个缺陷的两个实例,在这款产品自己的缺陷清单上作为一条而不是两条互不相干的怪癖记录。

除这两个之外,可写的参数槽位接受其槽位的全宽取值:固件对标识符作范围检查,而不是对取值。凡是某个取值必须合理设备才能正常动作的地方,那项约束属于产品,并且它在该产品自己的文档中。

A.4 遥测字段

第一部分 §7.2 确定了哪一位对应哪个物理量,并把两件事留给产品:每个字段背后的标度,以及到底哪些位接到了真实读数上。在这款产品上每个字段都是 u16,标度为:

- LOG_FIELD_VDD、LOG_FIELD_VAT —— 毫伏。
- LOG_FIELD_IAT —— 毫安。
- LOG_FIELD_PWR —— 毫瓦,由实测的负载电压、电流与驱动占空比推导而来,不是直接测量的。
- LOG_FIELD_DUTY —— 百分比。
- LOG_FIELD_RAT —— 毫欧,由设备计算而来,不是测量的。
- LOG_FIELD_PROT —— 设备保护状态的位域,不是一个带标度的量。

接到真实读数上的有:VDD、VAT、IAT、PWR、DUTY,以及在带干烧支持的恒功率构建上的 RAT。PROT 没有接线,而 RAT 在恒压构建上读作零,因为那种构建设没有可据以计算它的电流采样通道。未接线的位仍然会被接受,仍然在帧中占据它那个 u16,报告为零,与 §7.2 的要求完全一致。

A.5 调校标识符

第一部分 §6.4 确定了会话的形态,并把标识符留给产品。这一款定义了四个,而 CMD_TUNING_START 对当前构建不支持的标识符以 ERR_BAD_ARGS 拒绝:

- TUNING_ID_CV_VATRMS(0)——扫描恒压驱动目标。仅限恒压构建。
- TUNING_ID_BATTERY_GAUGE(1)——扫描电量计的电压分档。它是在一个放电阶段和一个充电阶段上定义的,而**出货固件只运行充电阶段**。一次会话走哪个阶段由设备决定,依据是充电器是否真的在位;若不在位,该会话本会是一次放电扫描,而 CMD_TUNING_START 直接以 ERR_NOT_READY 拒绝该标识符。这次拒绝是刻意的:放电扫描的那些帧会报告负载下的电压而不是电芯自身的电压,于是这次扫描会返回一批看上去像标定结果、实则不是的数字,而拒绝才是诚实的答案。其成因是这款产品测

量 通路上的一处回归,记录在它自己的缺陷清单上。这次拒绝不是下文那种预置拒绝 —— 它不取决于主机预置了什么 —— 二者只是共用同一个错误码。

- TUNING_ID_PROTECT_THRESH(2)—— 扫描短路线圈与断路线圈的电阻门限。
- TUNING_ID_DRYPUFF(3)—— 扫描干烧的温度门限与斜率门限。仅限带干烧支持的恒功率构建。

一次会话通过与 A.2 相同的外部控制输入通路运行真实的量产控制算法,因此主机在启动 一次会话之前先预置激励;预置的做法随那个接口一同记载,在该产品自己的文档中。对于预置的激励不可能产出任何测量结果的会话,CMD_TUNING_START 以 ERR_NOT_READY 拒绝,而不是跑完全程却什么也报不出来。

有一个标识符还会约束所请求的报告周期。电量计会话按它自己固定的采样节奏每周期报告一次,因此周期已无可控制之处:CMD_TUNING_START 对其他任何取值以 ERR_BAD_ARGS 拒绝,而不是接受下来再以另一个速率报告,因为请求了一个周期却悄悄 得到另一个周期的主机,会把时间戳之间的每一个间隔都读错。该节奏本身是这款产品的一个常量,随该产品自己的文档一同给出。除 §6.4 所要求的之外,没有别的标识符再约束周期。

索引元组表示什么。 §6.4 把 CMD_SET_TUNING_PAR/CMD_GET_TUNING_PAR 各索引 的含义留给产品。在这里,tuning_id 选中标定表,其余索引寻址表中的一个单元;某个标识符只使用它自己那张表具备维度的那些索引,而它所使用的每一个索引都会针对真实维度作边界检查,越界按 §6.4 的要求应答 ERR_BAD_ARGS。这两个命令都不需要会话。

- TUNING_ID_CV_VATRMS 是唯一用满整个元组的:coil_idx 为加热线圈, heating_mode 为驱动模式,heater_type 为所装配的发热体,step 为目标电压的 档位。
- TUNING_ID_BATTERY_GAUGE 把 coil_idx 用作**充放电方向** —— 放电或充电,不是 某根物理线圈 —— 并把 step 用作电量分档。
- TUNING_ID_PROTECT_THRESH 把 coil_idx 用作加热线圈,把 step 用来选择这一 对门限中被寻址的是哪一个:短路线圈门限还是断路线圈门限。
- TUNING_ID_DRYPUFF 把 step 用来选择绝对温度门限还是斜率门限;这些不是按线圈 划分的,但 coil_idx 仍会针对线圈数目作检查,因此点名了一根并不存在的线圈的主 机会被告知,而不是被照单服务。

某个标识符不使用的索引会被忽略,并不要求它为零,也不作范围检查。主机宜为它发送 零,因为日后赋予该维度含义的产品会开始检查它。

一次会话挂起哪些保护 —— 这是 §6.4 所要求的披露:只要主机在驱动加热触发,长 吸保护与加热超时保护就被挂起,这既涵盖单纯的外部控制,也涵盖调校会话;充电超时 保护在电量计会话期间被挂起,因为跨电芯全量程充电正是那次测量本身;而线圈电阻检 查在保护门限会话期间被挂起,那次扫描正是必须触及该检查存在的意义所在的那些取值 的一次。欠压锁定与充电过压保护全程保持有效。曾指望长吸保护或加热 超时保护来终止 一次过长的、由外部驱动的加热请求的主机,不再拥有那道后备。

那些挂起中有一处并不像它读起来那样触及得远。挂起线圈电阻检查,挂起的是设备在加 热期间运行的那项检查,而不是它在加热开始时运行的那一项:后者直接依据一次引 脚读数抬起断路线圈故障,其通路上没有任何东西是那次挂起触及得到的。因此,在一个 该检查本会拒绝的负载上运行的保护门限会话,仍然可能在加热开始时被拦下 —— 而这恰 恰是那次挂起存在所要允许的情形。它记录在这款产品自己的缺陷清单上,而披露了某项 挂起的产品宜使该挂起触及该披露所涵盖的每一项检查。

这款产品的协议表面上有一处不一致值得陈述,因为它是主机可见的:在恒压调校构建上, 那两个干烧门限槽位经由 CMD_PAR16_SET 是可写的,该命令应答 OK;而 CMD_SET_TUNING_PAR 拒绝它们,应答 ERR_BAD_ARGS,

其理由是正确的——在那种构 建中没有任何东西读取它们。两个操作码,一个字段,相反的答案。这是对 §8.2 的一处 偏离,而 §8.2 要求触及同一字段的两个操作码就它是否可写达成一致。它只是协议表面 上的不一致——无论如何那个字段都存在——而且它是被记录下来,尚未解决。

B. CVS-BP2601

同一标准之上的第二款产品。它的标识符映射表是它自己的,随那款产品一同保管;§8.1 完整适用于它们,而附录 A 不可被读作在描述它们。

与附录 A 之间在案的差异恰好只有一处,也正是 A.1 铺垫的那一处。 **CVS-BP2601 上的 CMD_SET_MODE 不触及日志使能参数。** 在那里进入 OPMODE_DEBUG 之后日志仍然是关闭的,直到一条明确的 CMD_LOG_START 为止;而 CFS-ECIG-SUITE 则随模式把它强制关闭和打开。两种行为都符合本标准——§5 定义的是一种模式是为什么而设,并没有把某个参数槽位绑到它上面——因此,想从任一款产品拿 到日志输出的主机,应明确地提出要求,而不是从自己所选的模式去推断。 CFS-ECIG-SUITE 的那一行为是该项目在 2026-07-10 作出的一个刻意选择,并且没有被回 移。

C. 桥接层

桥接器就是 §1.3 中的那个中继,而 §9.2 已经从设备一侧陈述了规则:0xc0 及以上属 于这一层,产品不得在那里定义命令,而桥接器仅依据操作码字节路由。本附录就是那条 规则为之保留了该区段的那张表。

ID	Name
0xC0	CMD_B_PING
0xC1	CMD_B_INFO
0xC2	CMD_B_GET_CAPS
0xC3	CMD_B_GET_STATS
0xC4	CMD_B_GET_SNAPSHOT
0xC5	CMD_SET_SWD_SPEED
0xC6	CMD_SET_TARGET
0xC7	CMD_B_GET_VERSION
0xC8	CMD_MODE_DEBUG
0xC9	CMD_MODE_DOWNLOAD
0xCA	CMD_MODE_RAW
0xCB	CMD_MODE_BRIDGE_RESET
0xCC	CMD_TGT_NRST
0xCD	CMD_TGT_POWER
0xCE	CMD_BRIDGE_CONF
0xD0	CMD_SWD_CONNECT

ID	Name
0xD1	CMD_SWD_DISCONNECT
0xD2	CMD_SWD_LINE_RESET
0xD3	CMD_SWD_JTAG2SWD
0xD4	CMD_SWD_READ_IDCODE
0xD7	CMD_SWD_CONNECT_UNDER_RESET
0xD8	CMD_DP_READ
0xD9	CMD_DP_WRITE
0xDA	CMD_AP_READ
0xDB	CMD_AP_WRITE
0xE0	CMD_CORE_HALT
0xE1	CMD_CORE_RESUME
0xE2	CMD_CORE_STATUS
0xE3	CMD_CORE_SYSRESET
0xE4	CMD_CORE_RUN_DETACH
0xE8	CMD_UPD_BEGIN
0xE9	CMD_UPD_DATA
0xEA	CMD_UPD_END
0xEB	CMD_UPD_STATUS
0xEC	CMD_UPD_ABORT
0xED	CMD_UPD_APPLY
0xF0	CMD_FLASH_ERASE_SECTOR
0xF1	CMD_FLASH_PROG_PAGE
0xF2	CMD_FLASH_PROG_PAGE_VERIFY
0xF3	CMD_FLASH_READ
0xF4	CMD_FLASH_MASS_ERASE
0xF5	CMD_FLASH_READ_EX
0xF8	CMD_OB_READ
0xF9	CMD_OB_WRITE
0xFA	CMD_OB_LAUNCH
0xFB	CMD_OB_WRITE_AND_LAUNCH

把它读作一层的表,而不是一块板子的表。目前有两块桥接板在役;它们的差别在性能,不在它们应答什么,而且两者都声明这张表。正是这一点使 §9.2 的保留成为一项保留,而不是某一块板子的约定俗成。

主机在使用它之前应知道四件事:

- **有些条目是保留的、没有处理函数**,而表中并不标出是哪些。某个操作码有没有处理函数,是作出应答的那份映像的属性,不是该区段这张表的属性:它随固件版本而变,而且引导加载程序应答的操作码远少于应用程序。保留的操作码像任何其他未实现的操作码一样被应答 ERR_BAD_ARGS(\$9.1),因此主机是去探测它,而不是先去读一个版本号。
- **CMD_UPD_*(0xE8-0xED)更新的是桥接器自身,而且只有它的引导加载程序应答它们。**它们不是 CMD_FLASH_* 区块,后者编程的是桥接器背后的那台设备。把两者搞混是一个在协议层面无从防范的错误:两个区块都在这同一个区段里,而且两个都会给出看似合理的应答。它们的形态是:CMD_UPD_BEGIN 接受一个固定 512 字节的映像头并应答 [OK][chunk_max u16][capacity u32];CMD_UPD_DATA 接受 [off u32][data],数据字节最多 256 个且绝不超过刚刚返回的 chunk_max,应答 [OK][next_off u32];CMD_UPD_STATUS 不带载荷,应答 [OK] 后跟一个固定 18 字节的状态块,其字段布局是桥接器自己的,随它一同记载;而 CMD_UPD_END、CMD_UPD_ABORT 和 CMD_UPD_APPLY 都不带载荷并应答 [OK],板子在应答完 CMD_UPD_APPLY 之后再复制映像并复位自己。那些尺寸是这一区块的线路常量,在每一块桥接板上都相同。
- **CMD_BRIDGE_CONF 是一个操作码承载许多项设置**,以一个参数字节作为键,因为这个区段可用的编号不多,而一个开关不该占掉一个。未知的参数被应答 ERR_BAD_ARGS,主机正是这样在不读版本号的情况下检出某项设置是否受支持。
- **CMD_B_PING 和 CMD_B_INFO 之所以存在,是因为设备自己的 CMD_PING 和 CMD_INFO 说不出是谁作的应答。**两层对那两个命令的应答方式相同,因此想确认桥接器在位的主机,要到桥接器自己的区段里去问。

正如 §6.1 所提醒的,桥接器的版本命令带有它自己的选择子空间:同一个选择子编号在那里表示的东西不同于设备端。除固件版本之外,它还应答当前运行映像的**角色**,用以区分应用程序与引导加载程序——这是主机最先该问的一件事,因为单凭一个固件版本说不出是哪份映像产出了它——以及板子的**硬件身份**,这是唯一——一个两份映像必须给出相同答案的选择子,因为映像变了硬件并不会变。

角色选择子也是主机据以知道该对命令集选择子有何预期的地方。桥接器的引导加载程序对那个选择子应答它自己的编号,并刻意冻结在低于应用程序的位置:它命名的是最后一次改动了引导加载程序自身所实现的那少数几个操作码的修订版,并且它不随应用程序的编号推进。刚刚读到角色并发现对方是引导加载程序的主机,应当预期接下来读到的命令集版本会更低,并且不应把这一差异当作故障或构建有误的发布。§9.3 的发布记账规则针对的是同一份映像那个常量的各份副本,而引导加载程序是另一份映像。

桥接器的错误码位于 §4.4 的扩展空间中,§4.4 点了它们各区块的名。有一种形态值得在此补充:每一个 ERR_UPD_* 码在码字节之后都携带一个 u32 细节,因为实机测试台上有用的问题不是一次更新是否失败,而是它失败在哪个偏移、哪个地址,或哪一项检查上。

C.1 CMSIS-DAP 人格

桥接器收到目标值为 0x02(BRIDGE_RESET_TO_DAP)的 CMD_MODE_BRIDGE_RESET 后,先应答,再复位,然后重新枚举为 CMSIS-DAP v1 调试单元:仅 USB HID,VID 0x0A05, PID 0x050B,一个 64 字节 IN 报告和一个 64 字节 OUT 报告,产品字符串中含有 CMSIS-DAP,iSerialNumber 与 CDC 人格相同。CMSIS-DAP 命令本身是 ARM 的 (CMSIS-DAP 2.1.1 命令集,仅 SWD,0x00-0x13);本标准不再复述它们。

供应商命令 0x80(DAP_Vendor0)是 CFS 隧道请求报告 [0x80][n][n 字节]($n \leq 62$)携带 CDC 人格所讲的同一个 COBS 字节流的接下来 n 个字节,而应答报告 [0x80][m][m 字节]返回应答流的接下来 m 个字节。 $n = 0$ 表示轮询更多应答字节。分帧、CRC 与操作码均不变。在此人格下,桥接器只应答 CMD_B_PING、CMD_B_INFO、CMD_B_GET_CAPS、CMD_B_GET_VERSION、CMD_B_GET_STATS、

CMD_B_GET_SNAPSHOT,以及 CMD_MODE_BRIDGE_RESET(所有目标 值);其余一切操作码,以及所有 DUT 操作码,均应答 ERR_NOT_READY。

这个人格保存在 RAM 中:断电重启总会让桥接器回到 CDC 人格,板上的 KEY 按钮也一样。

C.2 桥接器能力位(CMD_B_GET_CAPS)

自 2.0.0 起一直只是保留的 CMD_B_GET_CAPS(0xC2),从桥接命令集 3.0.0 起实现。请求不带载荷;应答为 [OK][caps u32 LE][reserved u32 LE]。第二个字为 0,主机忽略它。该命令不触碰任何引脚,因此在任一模式、任一人格下都会应答,主机可以在其他一切之前先发送它。

caps 说的是应答的固件能做什么,而 CMD_B_GET_VERSION(VER_HW) 说的是它跑在哪块板上。主机按这些位分支,而不是按自己维护的一张按板列出的表。各位永不重新分配;第 8-31 位未分配,读为 0。

- BCAP_TGT_NRST(0x00000001)—— CMD_TGT_NRST 驱动一条真实的 nRESET 线。
- BCAP_TGT_POWER_OEN(0x00000002)—— CMD_TGT_POWER 存在:桥接器能开关 DUT 的供电(BRD02 上该引脚本身就是电源轨,写入受 BCONF_TGT_POWER_EN 门控;BRD01 上由一只旁路晶体管门控 USB VBUS)。
- BCAP_PIN_MAP(0x00000004)—— BCONF_PIN_MAP 可在运行时选择引脚映射。
- BCAP_FLASH_READ_EX(0x00000008)—— 已实现 CMD_FLASH_READ_EX。
- BCAP_DAP_PERSONA(0x00000010)—— BRIDGE_RESET_TO_DAP 会把桥接器重启为 C.1 的 CMSIS-DAP HID 人格。
- BCAP_BTN_HW_RESET(0x00000020)—— 板上有硬件复位按钮(KEY)。
- BCAP_TGT_POWER_5V(0x00000040)—— 被开关的 DUT 供电为 5.0 V;为 0 则是 3.3 V IO 电源轨。
- BCAP_GPIO_CTRL(0x00000080)—— 为桥接器 GPIO 控制命令保留。目前尚无此命令,所以每块板都应答 0。

按构建结果,BRD02 应答 0x3F(除 5 V 电源轨外全部具备:它开关的是 3.3 V IO 轨), BRD01 应答 0x4B(nRESET、可开关供电、CMD_FLASH_READ_EX 与 5 V 轨;没有运行时引脚映射、没有 DAP 人格、没有 KEY)。

低于 3.0.0 的应用对未知操作码应答 ERR_BAD_ARGS,桥接器引导加载程序对未实现的操作码应答其默认的 ERR_NOT_READY。主机把任何错误应答都当作 caps = 0:桥接器说不出来,于是不假定任何可选功能存在。

C.3 闪存单位属于目标(CMD_SET_TARGET)

桥接器以目标设备自己的单位对其后的设备进行编程与擦除,而这个单位因系列而异: CWM0508、CWM0524、CWM1016 与 CWM2032 系列以 128 字节页编程、以 4 KB 扇区擦除;CWM30C8 系列(型号 0x04,命令集 3.1.0)和 CWM25C8 系列(型号 0x05,命令集 4.1.0)以 256 字节页编程、以 8 KB 扇区擦除。因此闪存块的三个操作码本身不带任何尺寸:

- CMD_FLASH_ERASE_SECTOR(0xF0)接受 [addr u32],擦除包含 addr 的那个扇区,由桥接器向下对齐;
- CMD_FLASH_PROG_PAGE(0xF1)与 CMD_FLASH_PROG_PAGE_VERIFY(0xF2)接受 [addr u32],其后恰好跟随所选目标的一页。携带其他任何字节数的帧应被应答 ERR_BAD_ARGS。校验形式的错误应答为 [err_code][mismatch_off u8],即页内第一个不一致字节的偏移。

主机从桥接器而不是从自己的表中得知这些单位:自 3.1.0 起,CMD_SET_TARGET 的应答在已经应答的闪存基址与大小之后、偏移 20 处携带 [page_size u16 LE][sector_size u16 LE]。低于 3.1.0 的桥接器应

答 20 字节,且不认识任何单位不是 128 B / 4 KB 的系列,所以在 20 处停止读取的主机在这样的桥接器上什么也不会损失;而选择了桥接器不认识的型号的主机,在发出任何一页之前就会收到 ERR_BAD_ARGS,这正是让 256 字节页永远到不了只编程 128 字节的桥接器的机制。

在 3.1.0 之前这三个操作码名为 CMD_FLASH_ERASE_4K、CMD_FLASH_PROG_PAGE_128 与 CMD_FLASH_PROG_PAGE_128_VERIFY。编号没有移动;只是名字不再宣称某个尺寸。

C.4 设备传输层:UART 或 RTT(BCONF_DUT_TRANSPORT)

默认情况下,桥接器通过 UART 把设备频段(0x00-0xBF)中继给设备——也就是桥接器不驱动 SWD 时那两根 SWD 共享引脚所承载的线路。从命令集 4.0.0 起,桥接器可以(MAY)为同一频段提供第二种传输层,由 CMD_BRIDGE_CONF 的 BCONF_DUT_TRANSPORT 参数在运行时选择:

- 取值 0x00(DUT_TRANSPORT_UART)——一如既往的 UART 中继。这是默认值,也是任何复位之后的状态。
- 取值 0x01(DUT_TRANSPORT_RTT)——桥接器在其处于中继模式期间始终保持一个 SWD 会话打开,并在设备内核运行的同时,经调试 MEM-AP 把同样的帧通过设备 RAM 中的一对环形缓冲区搬运。控制块的布局与位置与 SEGGER RTT 完全相同(16 字节 id 字符串,随后是一个 "up" 环与一个 "down" 环),因此普通的 RTT 主机工具也能看到它。所承载的帧与 UART 会承载的帧逐字节相同;传输层之上的一切都不改变。

RTT 是为那种没有 UART 外设、因而无法以链路比特率进行软件接收的设备而存在的;而每一颗 Cortex-M 设备都有 SWD,并且在这些板子上设备的 UART 引脚就是它的 SWD 引脚,所以这种传输层从不需要在两种角色之间倒换这些引脚。

该参数的应答——且仅有该参数——在每个 CMD_BRIDGE_CONF 应答都会返回的 [par][val] 之后再携带两个字段:

```
[OK][par][val][link u8][cb_addr u32 LE]
```

link 为 0 空闲(传输层是 UART)、1 down(已选 RTT,但尚无 SWD 会话)、2 searching(会话已建立,桥接器正在设备 RAM 中扫描控制块)或 3 up(已找到控制块并在中继)。cb_addr 在 link 为 3 时是该控制块的地址,否则为 0。只认识 [par][val] 的主机读取那两个字节并忽略其余。

当已选 RTT 传输层但链路尚未建立时,设备帧以 [ERR][ERR_NOT_READY][link u8] 应答——与中继模式无法投递一帧时给出的拒绝相同,只是在明细字节中带上链路状态,使主机能区分 "尚无设备" 与 "仍在搜索"。

低于 4.0.0 的桥接器、没有 RTT 引擎的桥接器,以及 CMSIS-DAP 人格,都会对该参数应答 ERR_BAD_ARGS,这正是主机探测其支持与否的办法。该设置不持久化:复位后的桥接器回到 UART 传输层。

D. DUT 测试固件

存在五份小固件,以便桥接器总有一台符合本标准的设备可以对话,桥接器所面对的每一颗芯片各一份:**CWM2032**、**CWM1016**、**CWM0508**、**CWM30C8** 和 **CWM25C8**。它们是测试目标,不是产品,不随任何东西出货。

每一份都恰好实现 §5.2 要求在 OPMODE_NORMAL 下被应答的那六个操作码 —— CMD_PING、CMD_INFO、CMD_GET_VERSION、CMD_SET_MODE、CMD_DBG_ONLINE 和 CMD_RESET —— 别的一概不实现,而且每一份都声明与各款产品相同的命令集版本,理由就是 §9.3 给出的那条:版本命名的是操作码表,不是它背后那个实现的规模。实现六个操作码、其余按 §9.1 应答的设备,与实现全部操作码的设备同样符合本标准。

E. 命令集变更历史

一个版本号覆盖两个区段(§9.3)。它在下列每一处重复出现:

- CFS-ECIG-SUITE 固件,及其主机工具;
- 两份桥接器固件各自一份、那两块板子的构建元数据文件各自一份,以及桥接器主机工具;
- 附录 D 中四份测试固件各自一份。

它们之中没有哪一份对其他各份具有权威性;各份互不一致的发布即为构建有误,这就是 §9.3 所陈述的发布记账规则。这份清单本身就是那个事实,而且清单旁边不写总数:§9.3 说过这些副本必须逐一数清而不能想当然,而写在清单旁边的一个总数是又一件需要维持为真的东西 —— 它正是会烂掉的那一件,而且下一次加板子的时候它还会再烂一次。去数这份清单。

有两样东西携带同一个常量,并被刻意排除在清单之外。一棵冻结的参考代码树携带它,因为它是一份用于比对的快照,不是产品。桥接器的各引导加载程序携带它们自己的一个编号,冻结且更低,理由见附录 C。

下面的每一条都是从那些副本的历史中推导出来的,而不是取自一份为此而维护的变更日志,而且每一条都点明了线路上有什么发生了变化。

- **2.0.0**(2026-08-21)—— 这个号码本身。两层开始一同声明一个命令集版本。设备侧 新增了 CMD_GET_VERSION(0x05),桥接侧新增了 CMD_B_GET_VERSION(0xC7),并且桥接器的两条配置命令被移到它的 SWD 门控之下,即它们本该在的位置,因为两者都不驱动任何 SWD 引脚。
- **2.1.0**(2026-08-26)—— CMD_FLASH_READ(0xF3)从保留变为已实现。
- **2.2.0**(2026-08-26)—— 桥接器自更新区块 CMD_UPD_*(0xE8-0xED),仅引导加载程序,连同它的 ERR_UPD_* 错误码;一个用于当前运行映像角色的版本选择子;以及 CMD_MODE_BRIDGE_RESET(0xCB)从保留变为已实现。目标值:0x00 应用、0x01 引导加载程序、0x02 CMSIS-DAP 人格(3.0.0,见 C.1;没有该人格的桥接器 应答 ERR_BAD_ARGS)。
- **2.3.0**(2026-08-27)—— CMD_BRIDGE_CONF(0xCE),一个操作码以参数/取值对的形式承载运行时设置,取代了每项设置一个操作码。
- **2.4.0**(2026-08-30)—— 桥接器版本命令上的一个硬件身份选择子,从任一份映像都能 应答板子的身份与版次。在那之前,区分两块桥接板的唯一办法是自由文本。
- **2.5.0**(2026-08-30)—— 内核暂停与内核恢复被给予它们自己的错误码,不再借用一个 闪存错误码,而那件事曾耗掉实机调试的时间(§4.4)。
- **2.6.0**(2026-09-01)—— 桥接器的选项字节命令中点名了一个选项字节掩码位。
- **2.7.0**(2026-09-03)—— 一个用于与复位线未接线的目标打交道的桥接器配置键。
- **2.8.0**(2026-09-04)—— 一个用于选择 SWD 引脚交换的桥接器配置键。
- **2.9.0**(2026-09-04)—— 引脚映射配置键,这是第一个其取值本身承载含义而不只是一个标志位的配置键,与之同时的还有主机侧的修正:此前主机会把配置取值压成零或一,那会把这个键的第二个取值当作它的第一个取值发出去。桥接器于是会选中第一个映射,并把它实际收到的那个取值原样回送,于是回复与到达的那一帧彼此自治,而线路上没有任何东西显示主机要的其实是另一个映射。

- **2.10.0**(2026-09-08)—— `CMD_FLASH_READ_EX(0xF5)`,一次同时返回普通读取无法报告的故障计数的读取。
- **2.11.0**(2026-09-09)—— 对 `CMD_TGT_POWER` 的写入被放到一个桥接器配置键之后,使得有能力为其目标供电的板子不再默认这样做。
- **3.0.0**(2026-09-11)—— `CMD_MODE_BRIDGE_RESET` 获得了第三个目标值 `BRIDGE_RESET_TO_DAP(0x02)`:桥接板不再重启为惯常的 CDC 人格,而是重启为 CMSIS-DAP v1 HID 调试单元。见 C.1。同时 `CMD_B_GET_CAPS(0xC2)`由保留变为已实现:一个能力字(`BCAP_*`),说明应答的固件能做什么,主机从此不必再维护按板列出的表。见 C.2。
- **3.1.0**(2026-09-13)—— `CMD_SET_TARGET` 接受第五个型号 `CWM30C8 (0x04)`,其应答携带目标的编程页与擦除扇区,因为该系列以 256 字节页编程、以 8 KB 扇区擦除,而此前每个系列都是 128 B / 4 KB。闪存块的三个操作码(`0xF0-0xF2`)编号不变,改名为说明其所做之事的名称——单位属于所选目标,而不属于操作码。见 C.3。附录 D 加入第四份测试固件。
- **4.0.0**(2026-09-15)—— 为设备频段提供第二种传输层: `BCONF_DUT_TRANSPORT` 在 UART 中继(默认)与经 SWD 穿过设备 RAM 的 RTT 式通道之间选择——为无法运行 UART 的设备而设。其应答携带链路状态与控制块地址,而因链路尚未建立被拒绝的设备帧会连同该状态以 `ERR_NOT_READY` 应答。见 C.4。此变更是可加的——UART 传输层不变,既有的每一次交换都逐字节相同——并非因为某个操作码移动或改变含义,而是出于对这一规模变更的项目决定,作为一次大版本发布。
- **4.1.0**(2026-09-15)—— 当前版本。`CMD_SET_TARGET` 接受第六个型号 `CWM25C8 (0x05)`。其闪存单位与 `CWM30C8` 系列相同(256 字节页、8 KB 扇区),应答为 3.1.0 的形式;线上其他内容不变。见 C.3。

记录到哪里为止。 它到 2.0.0 以下就没有了,而且无从追回:命令集版本是 2026-08-21 引入的,一引入就编号为 2.0.0,从来没有任何实现声明过 1.x。参考固件自己那份协议文档里出现的那些 v1.x 编号,是那份文档自己的修订号,它们不是命令集版本;把它们读作这份清单更早的条目会是一个错误。不过它们并非空无一物。那份文档在有线路可见的变更落地时才提升它的编号——增删或重编一条命令、改变一种帧布局、改变一项默认行为——而不为内部实现变更提升,而它的 v1.0 到 v1.5 各条记录的都是真实的线路变更:一个日志操作码挪了位置、突发载荷被重排、一种工作模式被废止、默认模式对未知命令的应答从一个错误变成了沉默,以及一条调校命令的载荷长出了一个前导字段。它们描述的是一款产品上的线路变更,而且完全早于命令集编号本身。

这份清单有两条属性值得直白地陈述。自 2.0.0 以来的每一次提升都是 MINOR,而且其中每一次都落在桥接区段里:设备区段自 2.0.0 以来没有变过,这就是为什么一台针对 2.0.0 构建的设备与一台针对 2.11.0 构建的主机,在这台设备所拥有的每一个操作码上仍然一致。这是关于已编号那一段的陈述,而不是关于更早时期的——在编号开始之前的那几周里,设备区段曾被反复重新编号,上面那些 v1.x 条目正记录了这一点,而这也是编号之所以存在的原因之一。并且并非每一次提升都是线路变更——2.9.0 有一部分是主机侧的记账——因为 §9.3 的规则是上面那份清单中的每一份副本一同变更,而不是每一次递增都增加一个操作码。